## Document Metadata

**Source:** https://github.com/Scoutflo/Scoutflo-SRE-Playbooks



**Generated:** 2026-06-10

**Total Playbooks:** 414 (232 K8s + 157 AWS + 25 Sentry)

# File: README.md

# SRE Playbooks Repository

[![License](https://img.shields.io/badge/license-MIT-blue.svg)](LICENSE) [![Contributions Welcome](https://img.shields.io/badge/contributions-welcome-brightgreen.svg)](CONTRIBUTING.md) [![GitHub Issues](https://img.shields.io/github/issues/Scoutflo/scoutflo-SRE-Playbooks)](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues) [![GitHub Stars](https://img.shields.io/github/stars/Scoutflo/scoutflo-SRE-Playbooks)](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/stargazers) [![GitHub Forks](https://img.shields.io/github/forks/Scoutflo/scoutflo-SRE-Playbooks)](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/network/members) [![GitHub Discussions](https://img.shields.io/github/discussions/Scoutflo/scoutflo-SRE-Playbooks)](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions) [![GitHub Contributors](https://img.shields.io/github/contributors/Scoutflo/scoutflo-SRE-Playbooks)](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/graphs/contributors)

> **Comprehensive incident response playbooks for AWS, Kubernetes, and Sentry environments** - Helping SREs diagnose and resolve infrastructure issues faster with systematic, step-by-step troubleshooting guides.

# Table of Contents



# Overview

This repository contains **414 comprehensive incident response playbooks** designed to help Site Reliability Engineers (SREs) systematically diagnose and resolve common infrastructure and application issues in AWS, Kubernetes, and Sentry environments.

## Why This Repository?

- **Systematic Approach**: Each playbook follows a consistent structure with clear diagnostic steps
- **Time-Saving**: Quickly identify root causes with correlation analysis frameworks
- **Community-Driven**: Continuously improved by the open-source community
- **Production-Ready**: Based on real-world incident response scenarios
- **Comprehensive Coverage**: 232 Kubernetes playbooks + 157 AWS playbooks + 25 Sentry playbooks
- **Proactive Monitoring**: 56 K8s + 65 AWS proactive playbooks for capacity planning and compliance

## Diagnosis Improvements

All playbooks use an **events-first approach** for root cause analysis:

- Diagnosis sections prioritize checking recent events and changes before diving into configuration details
- Conditional logic patterns help narrow down causes based on observed symptoms
- Time-based correlation analysis connects events to failures systematically

### Use Cases

- **During Incidents**: Quick reference for troubleshooting common issues
- **On-Call Rotation**: Essential runbook collection for on-call engineers
- **Knowledge Sharing**: Standardize troubleshooting procedures across teams
- **Training**: Learn systematic incident response methodologies
- **Documentation**: Build your own runbook library

## Repository Structure

```
scoutflo-SRE-Playbooks/
├── AWS Playbooks/                    # 157 AWS playbooks
│   ├── 01-Compute/                   # 27 playbooks (EC2, Lambda, ECS, EKS)
│   ├── 02-Database/                  # 8 playbooks (RDS, DynamoDB)
│   ├── 03-Storage/                   # 7 playbooks (S3)
│   ├── 04-Networking/                # 17 playbooks (VPC, ELB, Route53)
│   ├── 05-Security/                  # 16 playbooks (IAM, KMS, GuardDuty)
│   ├── 06-Monitoring/                # 8 playbooks (CloudTrail, CloudWatch)
│   ├── 07-CI-CD/                     # 9 playbooks (CodePipeline, CodeBuild)
│   ├── 08-Proactive/                 # 65 proactive monitoring playbooks
│   └── README.md
├── K8s Playbooks/                    # 232 Kubernetes playbooks
│   ├── 01-Control-Plane/             # 24 playbooks
│   ├── 02-Nodes/                     # 24 playbooks
│   ├── 03-Pods/                      # 41 playbooks
│   ├── 04-Workloads/                 # 25 playbooks
│   ├── 05-Networking/                # 27 playbooks
│   ├── 06-Storage/                   # 9 playbooks
│   ├── 07-RBAC/                      # 6 playbooks
│   ├── 08-Configuration/             # 6 playbooks
│   ├── 09-Resource-Management/       # 8 playbooks
│   ├── 10-Monitoring-Autoscaling/    # 3 playbooks
│   ├── 11-Installation-Setup/        # 1 playbook
│   ├── 12-Namespaces/                # 2 playbooks
│   ├── 13-Proactive/                 # 56 proactive monitoring playbooks
│   └── README.md
├── Sentry Playbooks/                 # 25 Sentry playbooks
│   ├── 01-Error-Tracking/            # 19 playbooks
```

```
│   ├── 02-Performance/              # 6 playbooks
│   ├── 03-Release-Health/           # Placeholder
│   └── README.md
├── CONTRIBUTING.md
└── README.md
```

# Contents

## AWS Playbooks (`AWS Playbooks/`)

**157 playbooks** covering 7 service categories + proactive monitoring:

- **Compute Services** (27 playbooks): EC2, Lambda, ECS, EKS
- **Database** (8 playbooks): RDS, DynamoDB
- **Storage** (7 playbooks): S3
- **Networking** (17 playbooks): VPC, ELB, Route 53, NAT Gateway
- **Security** (16 playbooks): IAM, KMS, GuardDuty, CloudTrail
- **Monitoring** (8 playbooks): CloudTrail, CloudWatch
- **CI/CD** (9 playbooks): CodePipeline, CodeBuild
- **Proactive** (65 playbooks): Capacity planning, compliance, cost optimization

**Key Topics:**

- Connection timeouts and network issues
- Access denied and permission problems
- Resource unavailability and capacity issues
- Security breaches and threat detection
- Service integration failures
- Proactive capacity and compliance monitoring

See [AWS Playbooks/README.md](AWS%20Playbooks/README.md) for complete documentation and playbook list.

## Kubernetes Playbooks (`K8s Playbooks/`)

**194 playbooks** organized into **13 categorized folders** covering Kubernetes cluster and workload issues:

**Folder Structure:**

- `01-Control-Plane/` (18 playbooks) - API Server, Scheduler, Controller Manager, etcd

- `02-Nodes/` (12 playbooks) - Node readiness, kubelet issues, resource constraints
- `03-Pods/` (31 playbooks) - Scheduling, lifecycle, health checks, resource limits
- `04-Workloads/` (23 playbooks) - Deployments, StatefulSets, DaemonSets, Jobs, HPA
- `05-Networking/` (19 playbooks) - Services, Ingress, DNS, Network Policies, kube-proxy
- `06-Storage/` (9 playbooks) - PersistentVolumes, PersistentVolumeClaims, StorageClasses
- `07-RBAC/` (6 playbooks) - ServiceAccounts, Roles, RoleBindings, authorization
- `08-Configuration/` (6 playbooks) - ConfigMaps and Secrets access issues
- `09-Resource-Management/` (8 playbooks) - Resource Quotas, overcommit, compute resources
- `10-Monitoring-Autoscaling/` (3 playbooks) - Metrics Server, Cluster Autoscaler
- `11-Installation-Setup/` (1 playbook) - Helm and installation issues
- `12-Namespaces/` (2 playbooks) - Namespace management issues
- `13-Proactive/` (56 playbooks) - Proactive monitoring, capacity planning, compliance

**Key Topics:**

- Pod lifecycle issues (CrashLoopBackOff, Pending, Terminating)
- Control plane component failures
- Network connectivity and DNS resolution
- Storage and volume mounting problems
- RBAC and permission errors
- Resource quota and capacity constraints
- Proactive capacity and compliance monitoring

See [K8s Playbooks/README.md](K8s%20Playbooks/README.md) for complete documentation and playbook list.

### Sentry Playbooks (`Sentry Playbooks/`)

**25 playbooks** covering error tracking and performance monitoring:

**Folder Structure:**

- `01-Error-Tracking/` (19 playbooks) - Error capture, grouping, alerting, and debugging
- `02-Performance/` (6 playbooks) - Transaction monitoring, performance issues, tracing
- `03-Release-Health/` - Release tracking and health monitoring (placeholder)

**Key Topics:**

- Error capture and reporting issues
- Issue grouping and deduplication
- Alert configuration and routing

- Performance transaction monitoring
- SDK integration troubleshooting
- Release health tracking

  See [Sentry Playbooks/README.md](Sentry%20Playbooks/README.md) for complete documentation and playbook list.

## Getting Started

### Prerequisites

- Basic knowledge of AWS services, Kubernetes, or Sentry
- Access to AWS Console, Kubernetes cluster, or Sentry dashboard (for using playbooks)
- Git (for cloning the repository)

### Installation

#### Option 1: Clone the Repository

```
bash

# Clone the repository

git clone https://github.com/Scoutflo/scoutflo-SRE-Playbooks.git

# Navigate to the repository

cd scoutflo-SRE-Playbooks

# View available playbooks

ls AWS\ Playbooks/
ls K8s\ Playbooks/
ls Sentry\ Playbooks/
```

#### Option 2: Use as Git Submodule

Include playbooks in your own projects:

```
bash
git submodule add https://github.com/Scoutflo/scoutflo-SRE-Playbooks.git playbooks
```

#### Option 3: Download Specific Playbooks

Browse and download individual playbooks directly from GitHub web interface.

### Quick Start

1. **Identify Your Issue**: Determine if it's an AWS, Kubernetes, or Sentry issue 2. **Navigate to Playbooks**: - AWS issues -> `AWS Playbooks/` - K8s issues -> `K8s Playbooks/[category-folder]/` - Sentry issues -> `Sentry Playbooks/[category-folder]/` 3. **Find the Playbook**: Match your symptoms to a playbook title 4. **Follow the Steps**: Execute diagnostic steps in order 5. **Use Diagnosis Section**: Apply correlation analysis for root cause identification

### Learn More

- **Watch Tutorials**: Check our [YouTube channel](https://www.youtube.com/@scoutflo6727) for video walkthroughs and best practices
- **AI SRE Demo**: Watch the [Scoutflo AI SRE Demo](https://youtu.be/P6xzFUtRqRc?si=0VN9oMV05rNzXFs8) to see AI-powered incident response
- **Scoutflo Documentation**: Visit [Scoutflo Documentation](https://scoutflo-documentation.gitbook.io/scoutflo-documentation) for platform guides
- **Join the Community**: Connect with other SREs in our [Slack workspace](https://scoutflo.slack.com)

### Example Usage

**Scenario**: EC2 instance SSH connection timeout

1. Navigate to `AWS Playbooks/` 2. Open `Connection-Timeout-SSH-Issues-EC2.md` 3. Follow the Playbook steps, replacing `<instance-id>` with your actual instance ID 4. Use the Diagnosis section to correlate events with failures 5. Apply the identified fix

## Usage

## How Playbooks Work

**Important**: These playbooks are designed for **AI agents** using natural language processing (NLP). They use natural language instructions that AI agents interpret and execute using available tools (like AWS MCP tools, Kubernetes MCP tools, or kubectl).

**Example Playbook Step:**

- Natural Language: "Retrieve logs from pod `<pod-name>` in namespace `<namespace>` and analyze error messages"
- AI Agent Action: Interprets this instruction and uses appropriate tools to fetch and analyze pod logs

**For Manual Use:**

- While playbooks are optimized for AI agents, you can also use them manually
- The README files in each category folder include equivalent kubectl/AWS CLI commands for manual verification
- Replace placeholders with actual resource identifiers when following steps manually

## Playbook Structure

All playbooks follow a consistent structure:

1. **Title** - Clear, descriptive issue identification 2. **Meaning** - What the issue means, triggers, symptoms, root causes 3. **Impact** - Business and technical implications 4. **Playbook** - 8-10 numbered diagnostic steps in natural language (ordered from common to specific) 5. **Diagnosis** - Correlation analysis framework with time windows using events-first approach and conditional logic patterns

## Best Practices

- **For AI Agents**: Playbooks are optimized for AI interpretation - use natural language instructions
- **For Manual Use**: See category README files for equivalent kubectl/AWS CLI commands
- **Replace Placeholders**: All playbooks use placeholders (e.g., `<instance-id>`, `<pod-name>`) that must be replaced with actual values
- **Follow Order**: Execute steps sequentially unless you have strong evidence pointing to a specific step
- **Correlate Timestamps**: Use the Diagnosis section to correlate events with failures
- **Extend Windows**: If initial correlations don't reveal causes, extend time windows as suggested

## Placeholder Reference

**AWS Playbooks:**

- `<instance-id>`, `<bucket-name>`, `<region>`, `<function-name>`, `<role-name>`, `<user-name>`, `<security-group-id>`, `<vpc-id>`, `<rds-instance-id>`, `<load-balancer-name>`

**Kubernetes Playbooks:**

- `<pod-name>`, `<namespace>`, `<deployment-name>`, `<node-name>`, `<service-name>`, `<ingress-name>`, `<pvc-name>`, `<configmap-name>`, `<secret-name>`

**Sentry Playbooks:**

- `<project-slug>`, `<organization-slug>`, `<issue-id>`, `<transaction-name>`, `<release-version>`, `<environment>`

## Terminology & Glossary

Understanding the terms used in these playbooks will help you use them more effectively. For detailed glossaries, see:

- [AWS Terminology](AWS%20Playbooks/README.md#terminology--glossary)
- [Kubernetes Terminology](K8s%20Playbooks/README.md#terminology--glossary)

### Quick Reference

**SRE (Site Reliability Engineering)**

- A discipline combining software engineering and operations to build reliable systems.

**Playbook / Runbook**

- A step-by-step guide for diagnosing and resolving specific issues.

**Incident**

- An event that disrupts or degrades a service, requiring immediate attention.

**On-Call**

- Engineers available to respond to incidents outside normal business hours.

**MTTR (Mean Time To Recovery)**

- Average time to restore a service after an incident. Playbooks help reduce MTTR.

**Correlation Analysis**

- Finding relationships between events (like configuration changes) and symptoms (like service failures) by comparing timestamps.

**Root Cause**

- The underlying reason why an issue occurred, as opposed to just the symptoms.

**Placeholder**

- A value in playbooks (like `<instance-id>`) that you replace with your actual resource identifier.

**Diagnosis Section**

- Part of each playbook that helps you correlate events with failures using time-based analysis.

### Common Abbreviations

- **K8s**: Kubernetes (K + 8 letters + s)
- **SRE**: Site Reliability Engineering
- **MTTR**: Mean Time To Recovery
- **API**: Application Programming Interface
- **DNS**: Domain Name System
- **RBAC**: Role-Based Access Control
- **PVC**: PersistentVolumeClaim
- **HPA**: Horizontal Pod Autoscaler

---

**For detailed explanations of AWS and Kubernetes terms, see the respective README files above.**

## Quick Reference

Need a quick cheat sheet? Check out our [Quick Reference Card](QUICK_REFERENCE.md) for:

- One-page overview
- Common commands
- Quick lookup tables
- Essential links

## Troubleshooting Guide

Not sure which playbook to use? Use our [Troubleshooting Decision Tree](TROUBLESHOOTING_FLOWCHART.md) to:

- Quickly identify the right playbook
- Navigate by issue type
- Look up by error message or alert name

## Examples & Use Cases

See real-world scenarios in [EXAMPLES.md](EXAMPLES.md):

- Step-by-step examples
- Common workflows
- Success stories
- Best practices

## FAQ

Have questions? Check our [FAQ](FAQ.md) for answers to:

- General questions
- Usage questions
- Technical questions
- Contributing questions

## Video Tutorials

Learn how to use these playbooks effectively:

- **YouTube Channel**: [@scoutflo6727](https://www.youtube.com/@scoutflo6727) - Subscribe for tutorials and walkthroughs
- **AI SRE Demo**: [Watch Demo Video](https://youtu.be/P6xzFUtRqRc?si=0VN9oMV05rNzXFs8) - See Scoutflo AI SRE in action
- **Tutorials**: Step-by-step video guides on using playbooks
- **Best Practices**: Learn SRE incident response best practices

**Coming Soon**: Video tutorials for:

- How to use playbooks effectively
- Common troubleshooting scenarios
- Contributing to playbooks
- Advanced correlation analysis

## Roadmap

Check out our [ROADMAP.md](ROADMAP.md) to see:

- Planned features and new playbook categories
- Short-term and long-term goals
- How to suggest new features
- Release history

## Contributing

We welcome contributions from the community! Your contributions help make these playbooks better for everyone. See our [Contributors](CONTRIBUTORS.md) page to see who has helped build this project.

> **First-time contributor?** Start with our [Getting Started Guide](.github/GETTING_STARTED.md) for a quick onboarding experience, then look for issues labeled `good first issue`.

### How to Contribute

#### 1. Reporting Issues

Found a bug, unclear instruction, or have a suggestion?

1. **Check Existing Issues**: Search [GitHub Issues](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues) first 2. **Create a New Issue**: - Use clear, descriptive title - Describe the problem or suggestion - Include relevant service/component, error messages, or examples - Tag with appropriate labels ( `aws-playbook` , `k8s-playbook` , `sentry-playbook` , `bug` , `enhancement` , etc.)

#### 2. Improving Existing Playbooks

To fix or enhance existing playbooks:

1. **Fork the Repository**: Create your own fork 2. **Create a Branch**:

```
bash
   git checkout -b fix/playbook-name-improvement
```

3. **Make Your Changes**: - Follow the established playbook structure - Maintain consistency with existing formatting - Update placeholders and examples as needed 4. **Test Your Changes**: Verify the playbook is accurate and helpful 5. **Commit and Push**:

```
bash
   git add .
   git commit -m "Fix: Improve [playbook-name] with [description]"
   git push origin fix/playbook-name-improvement
```

6. **Create a Pull Request**: - Provide clear description of changes - Reference any related issues - Request review from maintainers

#### 3. Adding New Playbooks

To add a new playbook for an uncovered issue:

1. **Check for Duplicates**: Ensure a similar playbook doesn't already exist 2. **Follow the Structure**: Use existing playbooks as templates 3. **Choose the Right Location**: - AWS playbooks -> `AWS Playbooks/` - K8s playbooks -> Appropriate category folder in `K8s Playbooks/` - Sentry playbooks -> Appropriate category folder in `Sentry Playbooks/` 4. **Follow Naming Conventions**: - AWS: `<IssueOrSymptom>-<Component>.md` - K8s: `<AlertName>-<Resource>.md` - Sentry: `<IssueType>-<Component>.md` 5. **Include All Sections**: Title, Meaning, Impact, Playbook (8-10 steps), Diagnosis (5 correlations) 6. **Update README**: Add the new playbook to the appropriate README's playbook list 7. **Create Pull Request**: Follow standard contribution process

## Contribution Guidelines

- **Follow the Structure**: Maintain consistency with existing playbooks
- **Use Placeholders**: Replace specific values with placeholders
- **Be Specific**: Provide actionable, step-by-step instructions
- **Include Correlation**: Add time-based correlation analysis in the Diagnosis section
- **Test Thoroughly**: Ensure playbooks are accurate and helpful
- **Document Changes**: Clearly describe what you changed and why

## Review Process

1. All contributions require review from maintainers 2. Feedback will be provided within 2-3 business days 3. Address any requested changes promptly 4. Once approved, your contribution will be merged

See [CONTRIBUTING.md](CONTRIBUTING.md) for detailed contribution guidelines.

## Connect with Us

We'd love to hear from you! Here are the best ways to connect:

### Community Channels

- **Slack Community**: [Join our Slack workspace](https://scoutflo.slack.com) for real-time discussions
- **GitHub Discussions**: [Start a discussion](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions) for questions and ideas
- **GitHub Issues**: [Report bugs or request features](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues)
- **LinkedIn**: Follow [Scoutflo on LinkedIn](https://www.linkedin.com/company/scoutflo/) for updates and insights
- **Twitter/X**: Follow [@scout_flo](https://x.com/scout_flo) for latest news and announcements

### Feedback & Feature Requests

Have an idea for improvement or a new playbook topic?

- **GitHub Issues**: Create a [feature request](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=feature_request.md)
- **Slack**: Share your ideas in our `#playbooks` channel

### Bug Reports

Found a bug or error in a playbook?

- **GitHub Issues**: Create a [bug report](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=bug_report.md)
- **Slack**: Report in our `#playbooks` channel for quick response

### Scoutflo Resources

- **Official Documentation**: [Scoutflo Documentation](https://scoutflo-documentation.gitbook.io/scoutflo-documentation) - Complete guide to Scoutflo platform
- **Website**: [scoutflo.com](https://scoutflo.com/) - Learn more about Scoutflo
- **AI SRE Tool**: [ai.scoutflo.com](https://ai.scoutflo.com/get-started) - AI-powered SRE assistant
- **Infra Management Tool**: [deploy.scoutflo.com](https://deploy.scoutflo.com/) - Kubernetes deployment platform
- **YouTube Channel**: [@scoutflo6727](https://www.youtube.com/@scoutflo6727) - Tutorials and demos
- **AI SRE Demo**: [Watch Demo Video](https://youtu.be/P6xzFUtRqRc?si=0VN9oMV05rNzXFs8) - See Scoutflo AI SRE in action
- **Blog**: [scoutflo.com/blog](https://scoutflo.com/blog) and [blog.scoutflo.com](https://blog.scoutflo.com/) - Latest articles and insights
- **Pricing**: [scoutflo.com/pricing](https://scoutflo.com/pricing) - Pricing information

### Additional Resources

- **Roadmap**: Check out our [project roadmap](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/projects) to see what's coming
- **Documentation**: Visit our [wiki](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/wiki) for detailed guides
- **Legal**: [Privacy Policy](https://blog.scoutflo.com/privacy/) | [Terms of Service](https://blog.scoutflo.com/terms/)

## Support

Need help? Check out our [Support Guide](.github/SUPPORT.md) or:

- **Questions**: [GitHub Discussions](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions)
- **Bugs**: [Report an issue](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=bug_report.md)
- **Features**: [Request a feature](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=feature_request.md)
- **Security**: See [SECURITY.md](SECURITY.md)

## Related Resources

### AWS Resources

**Official Documentation:**

- [AWS Documentation](https://docs.aws.amazon.com/) - Complete AWS service documentation
- [AWS Well-Architected Framework](https://aws.amazon.com/architecture/well-architected/) - Best practices for building on AWS
- [AWS Troubleshooting Guides](https://docs.aws.amazon.com/general/latest/gr/aws_troubleshooting.html) - Official troubleshooting guides
- [AWS Service Health Dashboard](https://status.aws.amazon.com/) - Check AWS service status

**Learning & Best Practices:**

- [AWS Architecture Center](https://aws.amazon.com/architecture/) - Reference architectures
- [AWS Security Best Practices](https://aws.amazon.com/security/security-resources/) - Security guidelines
- [AWS re:Post](https://repost.aws/) - AWS community Q&A
- [AWS Training](https://aws.amazon.com/training/) - Free and paid training courses

**Tools & Utilities:**

- [AWS CLI Documentation](https://docs.aws.amazon.com/cli/latest/userguide/) - Command-line interface
- [AWS CloudShell](https://aws.amazon.com/cloudshell/) - Browser-based shell
- [AWS Systems Manager](https://docs.aws.amazon.com/systems-manager/) - Operations management
- [AWS CloudWatch](https://docs.aws.amazon.com/cloudwatch/) - Monitoring and observability

## Kubernetes Resources

**Official Documentation:**

- [Kubernetes Documentation](https://kubernetes.io/docs/) - Complete Kubernetes documentation
- [kubectl Cheat Sheet](https://kubernetes.io/docs/reference/kubectl/cheatsheet/) - Quick command reference
- [Kubernetes Troubleshooting](https://kubernetes.io/docs/tasks/debug/) - Official troubleshooting guide
- [Kubernetes API Reference](https://kubernetes.io/docs/reference/kubernetes-api/) - API documentation

**Learning & Best Practices:**

- [Kubernetes Best Practices](https://kubernetes.io/docs/concepts/cluster-administration/) - Cluster administration
- [Kubernetes Security Best Practices](https://kubernetes.io/docs/concepts/security/) - Security guidelines
- [CNCF Cloud Native Trail Map](https://github.com/cncf/trailmap) - Learning path
- [Kubernetes.io Blog](https://kubernetes.io/blog/) - Latest updates and tutorials

**Tools & Utilities:**

- [k9s](https://k9scli.io/) - Terminal UI for Kubernetes
- [Lens](https://k8slens.dev/) - Kubernetes IDE
- [Helm](https://helm.sh/) - Package manager for Kubernetes
- [kubectx & kubens](https://github.com/ahmetb/kubectx) - Context and namespace switching

**Community Resources:**

- [Kubernetes Slack](https://slack.k8s.io/) - Community chat
- [Stack Overflow - Kubernetes](https://stackoverflow.com/questions/tagged/kubernetes) - Q&A
- [r/kubernetes](https://www.reddit.com/r/kubernetes/) - Reddit community
- [Kubernetes Forum](https://discuss.kubernetes.io/) - Discussion forum

## SRE Resources

**Books & Guides:**

- [Google SRE Book](https://sre.google/books/) - Site Reliability Engineering book
- [Site Reliability Engineering](https://sre.google/sre-book/table-of-contents/) - SRE practices
- [The Site Reliability Workbook](https://sre.google/workbook/table-of-contents/) - Practical SRE guide
- [Building Secure & Reliable Systems](https://sre.google/books/building-secure-reliable-systems/) - Security and reliability

**Learning Resources:**

- [SRE Foundation Course](https://www.cncf.io/certification/training/) - CNCF training
- [SRE Weekly](https://sreweekly.com/) - Weekly newsletter
- [SREcon](https://www.usenix.org/conferences/byname/srecon) - SRE conferences
- [Incident Response Guide](https://response.pagerduty.com/) - PagerDuty's incident response guide

**Tools & Platforms:**

- [Prometheus](https://prometheus.io/) - Monitoring and alerting

- [Grafana](https://grafana.com/) - Visualization and dashboards
- [Jaeger](https://www.jaegertracing.io/) - Distributed tracing
- [ELK Stack](https://www.elastic.co/what-is/elk-stack) - Logging and analysis

### Incident Response & Runbooks

**Runbook Resources:**

- [PagerDuty Incident Response](https://response.pagerduty.com/) - Incident response best practices
- [Atlassian Incident Management](https://www.atlassian.com/incident-management) - Incident management guide
- [GitLab Runbooks](https://about.gitlab.com/handbook/engineering/infrastructure/runbooks/) - Example runbooks
- [Google's SRE Runbook Template](https://sre.google/workbook/runbooks/) - Runbook structure

**Incident Management:**

- [Incident.io](https://incident.io/) - Incident management platform
- [FireHydrant](https://www.firehydrant.com/) - Incident response platform
- [Statuspage](https://www.statuspage.io/) - Status page management

### Community & Forums

**General DevOps:**

- [DevOps Reddit](https://www.reddit.com/r/devops/) - DevOps community
- [DevOps Stack Exchange](https://devops.stackexchange.com/) - Q&A platform
- [HashiCorp Learn](https://learn.hashicorp.com/) - Infrastructure tutorials

**Cloud Native:**

- [CNCF Resources](https://www.cncf.io/) - Cloud Native Computing Foundation
- [Cloud Native Landscape](https://landscape.cncf.io/) - CNCF project landscape
- [CNCF Webinars](https://www.cncf.io/webinars/) - Educational webinars

## Statistics

- **Total Playbooks**: 376

- AWS: 157 playbooks (92 reactive + 65 proactive) - Kubernetes: 194 playbooks (138 reactive + 56 proactive) - Sentry: 25 playbooks
- **Coverage**: Major AWS services, Kubernetes components, and Sentry monitoring
- **Format**: Markdown with structured sections
- **Language**: English
- **Community**: Open source, community-driven

## License

This project is licensed under the MIT License - see the [LICENSE](LICENSE) file for details.

## Maintainers

This project is maintained by:

- [@AtharvaBondreScoutflo](https://github.com/AtharvaBondreScoutflo)
- [@Vedant-Vyawahare](https://github.com/Vedant-Vyawahare)

For maintainer information, see [MAINTAINERS.md](MAINTAINERS.md).

## Acknowledgments

- **Contributors**: Thank you to all contributors who help improve these playbooks
- **Community**: The SRE community for sharing knowledge and best practices
- **Organizations**: Companies and teams using these playbooks in production

---

**Made with love by the SRE community for the SRE community**

If you find these playbooks helpful, please consider giving us a star on GitHub!

# File: QUICK_REFERENCE.md

# Quick Reference Card

One-page cheat sheet for SRE Playbooks.

## 🚀 Quick Start

```
1. Identify Issue → AWS or Kubernetes?
2. Find Playbook → Match symptoms to title
3. Follow Steps → Replace placeholders
4. Use Diagnosis → Correlate events
5. Apply Fix → Document findings
```

## 📁 Repository Structure

```
scoutflo-SRE-Playbooks/
├── AWS Playbooks/          (157 playbooks in 8 folders)
│   ├── 01-Compute/         (27) - EC2, Lambda, ECS, EKS
│   ├── 02-Database/        (8)  - RDS, DynamoDB
│   ├── 03-Storage/         (7)  - S3
│   ├── 04-Networking/      (17) - VPC, ELB, Route53
│   ├── 05-Security/        (16) - IAM, KMS, GuardDuty
│   ├── 06-Monitoring/      (8)  - CloudTrail, CloudWatch
│   ├── 07-CI-CD/           (9)  - CodePipeline
│   └── 08-Proactive/       (65) - Proactive monitoring
├── K8s Playbooks/          (194 playbooks in 13 folders)
│   ├── 01-Control-Plane/   (18)
│   ├── 02-Nodes/           (12)
│   ├── 03-Pods/            (31) ⭐ Most common
│   ├── 04-Workloads/       (23)
│   ├── 05-Networking/      (19)
│   ├── 06-Storage/         (9)
│   ├── 07-RBAC/            (6)
│   ├── 08-Configuration/   (6)
│   ├── 09-Resource-Management/ (8)
│   ├── 10-Monitoring-Autoscaling/ (3)
│   ├── 11-Installation-Setup/ (1)
│   ├── 12-Namespaces/      (2)
│   └── 13-Proactive/       (56) - Proactive monitoring
└── Sentry Playbooks/       (25 playbooks in 3 folders)
    ├── 01-Error-Tracking/  (19)
    ├── 02-Performance/     (6)
    └── 03-Release-Health/  (placeholder)
```

## 🔍 Finding the Right Playbook

| Issue Type | Category | Example Playbook | |---|---|---| | Pod crashing | `03-Pods/` | `CrashLoopBackOff-pod.md` | | Pod not starting | `03-Pods/` | `PendingPods-pod.md` | | Service not working | `05-Networking/` | `ServiceNotAccessible-service.md` | | Permission denied | `07-RBAC/` | `RBACPermissionDeniedError-rbac.md` | | Volume mount failed | `06-Storage/` | `PVCPendingDueToStorageClassIssues-storage.md` | | Deployment not scaling | `04-Workloads/` | `DeploymentNotScalingProperly-deployment.md` | | Node not ready | `02-Nodes/` | `KubeNodeNotReady-node.md` | | API Server down | `01-Control-Plane/` | `KubeAPIDown-control-plane.md` | | EC2 SSH timeout | `AWS Playbooks/` | `Connection-Timeout-SSH-Issues-EC2.md` | | RDS connection failed | `AWS Playbooks/` | `Connection-Timeout-from-Lambda-RDS.md` | | Sentry error | `01-Error-Tracking/` | `UnhandledException-Error-application.md` | | API timeout | `02-Performance/` | `TimeoutError-RequestTimeout-API-Error-application.md` | | AWS cost issue | `08-Proactive/04-Cost-Optimization/` | `Cost-Anomaly-Detection-AWS.md` | | K8s capacity | `13-Proactive/01-Capacity-Performance/` | `Capacity-Trend-Analysis-K8s.md` |

## 🔧 Common Commands

### Kubernetes

```
bash
```

# Pods

```
kubectl get pods -n <namespace>
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace> --previous
```

# Services

```
kubectl get services -n <namespace>
kubectl describe service <service-name> -n <namespace>
```

# Nodes

```
kubectl get nodes
kubectl describe node <node-name>
```

## Events

```
kubectl get events -n <namespace> --sort-by='.lastTimestamp'
```

## Resource usage

```
kubectl top pods -n <namespace>
kubectl top nodes
```

### AWS

```
bash
```

## EC2

```
aws ec2 describe-instances --instance-ids <instance-id>
aws ec2 describe-security-groups --group-ids <sg-id>
```

## RDS

```
aws rds describe-db-instances --db-instance-identifier <id>
```

## Lambda

```
aws lambda get-function-configuration --function-name <name>
```

### CloudWatch Logs

```
aws logs tail <log-group-name> --follow
```

## 📝 Placeholder Reference

### AWS

- `<instance-id>` - EC2 instance ID
- `<bucket-name>` - S3 bucket name
- `<region>` - AWS region (e.g., us-east-1)
- `<function-name>` - Lambda function name
- `<vpc-id>` - VPC identifier

### Kubernetes

- `<pod-name>` - Pod name
- `<namespace>` - Kubernetes namespace
- `<deployment-name>` - Deployment name
- `<node-name>` - Node name
- `<service-name>` - Service name

## 🎯 Playbook Structure

Every playbook has: 1. **Title** - Issue identification 2. **Meaning** - What the issue means 3. **Impact** - Business/technical impact 4. **Playbook** - 8-10 diagnostic steps 5. **Diagnosis** - Correlation analysis

## ⚡ Quick Troubleshooting

### Pod Issues

```
Pod crashing? → 03-Pods/CrashLoopBackOff-pod.md
Pod pending? → 03-Pods/PendingPods-pod.md
Image pull failed? → 03-Pods/ImagePullBackOff-registry.md
```

### Network Issues

```
Service not accessible? → 05-Networking/ServiceNotAccessible-service.md
DNS not resolving? → 05-Networking/ServiceNotResolvingDNS-dns.md
Ingress not working? → 05-Networking/IngressNotWorking-ingress.md
```

### Resource Issues

```
Quota exceeded? → 09-Resource-Management/KubeQuotaExceeded-namespace.md
Node disk full? → 02-Nodes/NodeDiskPressure-storage.md
High CPU? → 09-Resource-Management/HighCPUUsage-compute.md
```

## 🔗 Essential Links

- **Main README**: [README.md](README.md)
- **Contributing**: [CONTRIBUTING.md](CONTRIBUTING.md)
- **FAQ**: [FAQ.md](FAQ.md)
- **Examples**: [EXAMPLES.md](EXAMPLES.md)
- **Support**: [.github/SUPPORT.md](.github/SUPPORT.md)

## 📚 Resources

- **Kubernetes Docs**: https://kubernetes.io/docs/
- **AWS Docs**: https://docs.aws.amazon.com/
- **Scoutflo Docs**: https://scoutflo-documentation.gitbook.io/scoutflo-documentation
- **YouTube**: https://www.youtube.com/@scoutflo6727

## 🆘 Getting Help

- **GitHub Discussions**: Ask questions
- **GitHub Issues**: Report bugs
- **Slack**: Join community
- **Email**: security@scoutflo.com (security issues)

## 💡 Pro Tips

1. **Bookmark** frequently used playbooks 2. **Clone locally** for offline access 3. **Customize** for your environment 4. **Contribute** improvements back 5. **Share** with your team

---

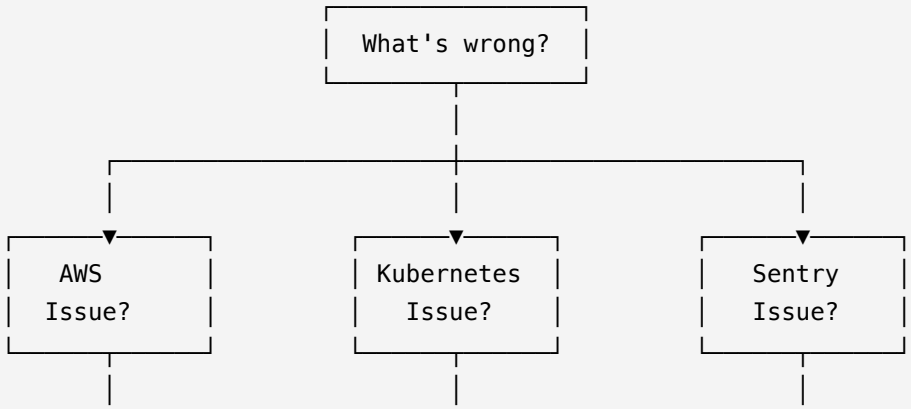
**Print this page** and keep it handy!

# File: TROUBLESHOOTING_FLOWCHART.md

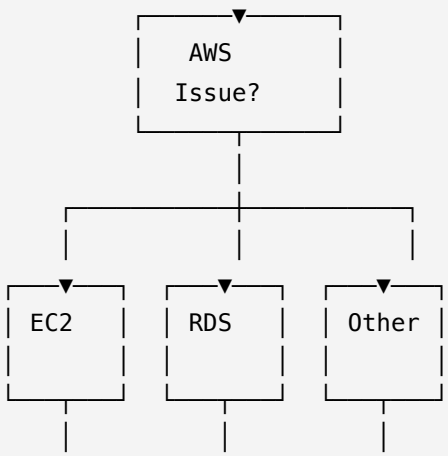
# Troubleshooting Decision Tree

Use this flowchart to quickly identify which playbook to use for your issue.

## Start Here: What Type of Issue?




## AWS Issues Path



```
    |         |         |
    ▼         ▼         ▼
```

## AWS Compute Issues (01-Compute/)

```
Compute Issue?
    |
    ├─ Can't SSH to EC2? → 01-Compute/Connection-Timeout-SSH-Issues-EC2.md
    ├─ EC2 not starting? → 01-Compute/Instance-Not-Starting-EC2.md
    ├─ Lambda timeout? → 01-Compute/Timeout-Error-Lambda.md
    ├─ ECS task pending? → 01-Compute/Task-Stuck-in-Pending-State-ECS.md
    └─ EKS pods crashing? → 01-Compute/Pod-Stuck-in-CrashLoopBackOff-EKS.md
```

## AWS Database Issues (02-Database/)

```
Database Issue?
    |
    ├─ Can't connect to RDS? → 02-Database/Instance-Not-Connecting-RDS.md
    ├─ RDS storage full? → 02-Database/Storage-Full-Error-RDS.md
    ├─ DynamoDB throttling? → 02-Database/Throttling-Errors-DynamoDB.md
    └─ Backup failing? → 02-Database/Automatic-Backup-Not-Working-RDS.md
```

## AWS Networking Issues (04-Networking/)

```
Networking Issue?
    |
    ├─ ELB not routing? → 04-Networking/Not-Routing-Traffic-ELB.md
    ├─ DNS failing? → 04-Networking/DNS-Resolution-Failing-Route-53.md
    ├─ API Gateway 500? → 04-Networking/Returning-500-Internal-Server-Error-API-Gateway.md
    └─ VPC peering? → 04-Networking/Peering-Not-Working-VPC.md
```

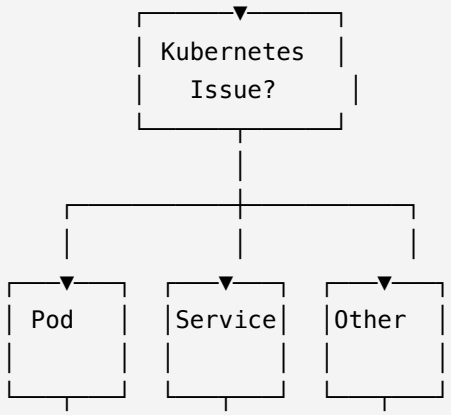
## AWS Security Issues (05-Security/)

```
Security Issue?
    |
    ├─ IAM permissions? → 05-Security/Policy-Not-Granting-Expected-Access-IAM.md
    ├─ Access key leaked? → 05-Security/Access-Key-Leaked-Warning-IAM.md
    ├─ KMS decryption? → 05-Security/Key-Policy-Preventing-Decryption-KMS.md
    └─ WAF blocking? → 05-Security/Blocking-Legitimate-Traffic-WAF.md
```

## Other AWS Issues

```
Other AWS Issue?
    │
    ├─ S3 access denied? → 03-Storage/Bucket-Access-Denied-403-Error-S3.md
    ├─ CloudWatch alarm? → 06-Monitoring/Alarm-Not-Triggering-as-Expected-CloudWatch.md
    ├─ Pipeline stuck? → 07-CI-CD/Stuck-in-Progress-CodePipeline.md
    └─ Proactive checks? → 08-Proactive/
```

# Kubernetes Issues Path




## Pod Issues (Most Common)

```
Pod Issue?
    │
    ├─ Crashing? → 03-Pods/CrashLoopBackOff-pod.md
    ├─ Not starting? → 03-Pods/PendingPods-pod.md
    ├─ Image pull failed? → 03-Pods/ImagePullBackOff-registry.md
    ├─ Stuck terminating? → 03-Pods/PodsStuckinTerminatingState-pod.md
    ├─ Health check failing? → 03-Pods/PodFailsLivenessProbe-pod.md
    └─ Can't access logs? → 03-Pods/PodLogsNotAvailable-pod.md
```

## Service/Network Issues

```
Service/Network Issue?
    │
    ├─ Service not accessible? → 05-Networking/ServiceNotAccessible-service.md
    ├─ DNS not resolving? → 05-Networking/ServiceNotResolvingDNS-dns.md
    ├─ Ingress not working? → 05-Networking/IngressNotWorking-ingress.md
    ├─ CoreDNS down? → 05-Networking/CoreDNSPodsCrashLooping-dns.md
    └─ Network policy? → 05-Networking/NetworkPolicyBlockingTraffic-network.md
```

## Other Kubernetes Issues

```
Other K8s Issue?
    │
    ├─ Node not ready? → 02-Nodes/KubeNodeNotReady-node.md
    ├─ Deployment not scaling? → 04-Workloads/DeploymentNotScalingProperly-deployment.md
    ├─ Volume mount failed? → 06-Storage/PVCPendingDueToStorageClassIssues-storage.md
    ├─ Permission denied? → 07-RBAC/RBACPermissionDeniedError-rbac.md
    ├─ ConfigMap not found? → 08-Configuration/ConfigMapNotFound-configmap.md
    ├─ Quota exceeded? → 09-Resource-Management/KubeQuotaExceeded-namespace.md
    ├─ API Server down? → 01-Control-Plane/KubeAPIDown-control-plane.md
    └─ HPA not scaling? → 04-Workloads/HPAHorizontalPodAutoscalerNotScaling-workload.md
```

# Detailed Decision Tree

## Step 1: Identify the Component

```
What component is affected?
    │
    ├─ Pod? → Go to Pod Decision Tree
    ├─ Service? → Go to Service Decision Tree
    ├─ Node? → Go to Node Decision Tree
    ├─ Deployment? → Go to Workload Decision Tree
    ├─ Volume? → Go to Storage Decision Tree
    ├─ Permission? → Go to RBAC Decision Tree
    └─ Control Plane? → Go to Control Plane Decision Tree
```

## Step 2: Identify the Symptom

#### Pod Decision Tree

```
Pod Issue?
    │
    ├─ State: CrashLoopBackOff → CrashLoopBackOff-pod.md
    ├─ State: Pending → PendingPods-pod.md
    ├─ State: ImagePullBackOff → ImagePullBackOff-registry.md
    ├─ State: Terminating → PodsStuckinTerminatingState-pod.md
    ├─ Health: Liveness probe failing → PodFailsLivenessProbe-pod.md
    ├─ Health: Readiness probe failing → PodFailsReadinessProbe-pod.md
    ├─ Logs: Not available → PodLogsNotAvailable-pod.md
    └─ Other → Browse 03-Pods/ folder
```

#### Service Decision Tree

```
Service Issue?
    │
```

```
    ├─ Not accessible → ServiceNotAccessible-service.md
    ├─ DNS not resolving → ServiceNotResolvingDNS-dns.md
    ├─ Not forwarding traffic → ServiceNotForwardingTraffic-service.md
    ├─ Connection refused → ErrorConnectionRefusedWhenAccessingService-service.md
    └─ Intermittent → ServicesIntermittentlyUnreachable-service.md
```

#### Node Decision Tree

```
Node Issue?
    │
    ├─ Not ready → KubeNodeNotReady-node.md
    ├─ Unreachable → KubeNodeUnreachable-node.md
    ├─ Disk pressure → NodeDiskPressure-storage.md
    ├─ Too many pods → KubeletTooManyPods-node.md
    ├─ Kubelet down → KubeletDown-node.md
    └─ Can't join cluster → NodeCannotJoinCluster-node.md
```

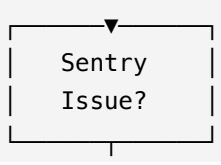
## Quick Lookup by Error Message

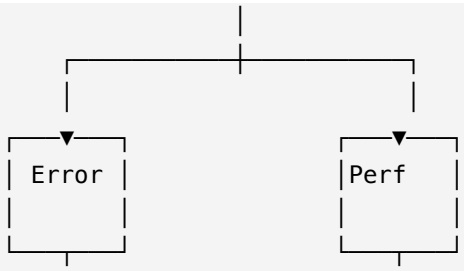
| Error Message | Playbook | |---|---| | `CrashLoopBackOff` | `03-Pods/CrashLoopBackOff-pod.md` || `ImagePullBackOff` | `03-Pods/ImagePullBackOff-registry.md` || `Pending` | `03-Pods/PendingPods-pod.md` || `NodeNotReady` | `02-Nodes/KubeNodeNotReady-node.md` | | `Forbidden` | `07-RBAC/ErrorForbiddenwhenRunningkubectlCommands-rbac.md` || `Connection refused` | `05-Networking/ErrorConnectionRefusedWhenAccessingService-service.md` || `Quota exceeded` | `09-Resource-Management/KubeQuotaExceeded-namespace.md` || `Volume mount failed` | `06-Storage/PodCannotAccessPersistentVolume-storage.md` |

## Quick Lookup by Alert Name

| Alert Name | Playbook | |---|---| | `KubePodCrashLooping` | `03-Pods/KubePodCrashLooping-pod.md` || `KubeNodeNotReady` | `02-Nodes/KubeNodeNotReady-node.md` || `KubeAPIDown` | `01-Control-Plane/KubeAPIDown-control-plane.md` || `KubeQuotaExceeded` | `09-Resource-Management/KubeQuotaExceeded-namespace.md` || `KubeDeploymentReplicasMismatch` | `04-Workloads/KubeDeploymentReplicasMismatch-deployment.md` |

## Sentry Issues Path






### Sentry Error Tracking Issues (01-Error-Tracking/)

```
Error Tracking Issue?
    │
    ├─ Database connection error? → 01-Error-Tracking/ConnectionError-ConnectionRefused-Databas
    ├─ Redis connection error? → 01-Error-Tracking/ConnectionError-ConnectionRefused-Redis-Erro
    ├─ Kafka consumer error? → 01-Error-Tracking/ConsumerError-ConnectionError-Kafka-Error-appl
    ├─ API call failed? → 01-Error-Tracking/APICallFailed-Error-application.md
    ├─ Unhandled exception? → 01-Error-Tracking/UnhandledException-Error-application.md
    ├─ Memory error? → 01-Error-Tracking/MemoryError-Error-application.md
    ├─ Key/Value error? → 01-Error-Tracking/KeyError-MissingKey-Error-application.md
    └─ Validation error? → 01-Error-Tracking/ValidationError-DataValidation-Error-application.m
```

### Sentry Performance Issues (02-Performance/)

```
Performance Issue?
    │
    ├─ Database timeout? → 02-Performance/TimeoutError-QueryTimeout-Database-Error-application.
    ├─ Redis timeout? → 02-Performance/ConnectionError-ConnectionTimeout-Redis-Error-applicatio
    ├─ API timeout? → 02-Performance/TimeoutError-RequestTimeout-API-Error-application.md
    └─ Connection timeout? → 02-Performance/TimeoutError-ConnectionTimeout-API-Error-applicatio
```

## Still Not Sure?

1. **Check the main README**: [Main README](README.md) 2. **Browse by category**: Each folder has a README explaining what it covers 3. **Search the repository**: Use GitHub's search or Ctrl+F 4. **Ask the community**: [GitHub Discussions](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions)

## Pro Tips

1. **Start with Pods**: Most issues manifest as pod problems 2. **Check Events**: `kubectl get events` often points to the right playbook 3. **Read Category READMEs**: Each folder README explains what it covers 4. **Use Search**: GitHub search is your friend

---

**Need help navigating?** Check the [Quick Reference Card](QUICK_REFERENCE.md) or [FAQ](FAQ.md)!

# File: FAQ.md

# Frequently Asked Questions (FAQ)

Common questions about using the SRE Playbooks repository.

## General Questions

### What are these playbooks?

These are step-by-step troubleshooting guides for common AWS, Kubernetes, and Sentry issues. Each playbook provides systematic diagnostic steps to help SREs and on-call engineers resolve infrastructure problems faster.

### How many playbooks are there?

- **376 total playbooks**
  - 157 AWS playbooks (organized in 8 categories) - 194 Kubernetes playbooks (organized in 13 categories) - 25 Sentry playbooks (organized in 3 categories)

### Are these playbooks free to use?

Yes! This is an open-source repository under the MIT License. You can use, modify, and distribute these playbooks freely.

### Can I contribute to these playbooks?

Absolutely! We welcome contributions. See our [Contributing Guide](CONTRIBUTING.md) for details on how to:

- Report bugs
- Improve existing playbooks
- Add new playbooks

## Using the Playbooks

### How do I find the right playbook for my issue?

1. **Identify the service**: Is it AWS, Kubernetes, or Sentry? 2. **Match symptoms**: Look for playbooks with titles matching your issue 3. **Check categories**: Browse the numbered category folders (8 for AWS, 13 for K8s, 3 for Sentry) 4. **Search**: Use GitHub's search or Ctrl+F to find keywords

### What if I can't find a playbook for my issue?

- **Create an issue**: Request a new playbook via [GitHub Issues](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=feature_request.md)
- **Contribute**: Create the playbook yourself following our [Contributing Guide](CONTRIBUTING.md)
- **Ask the community**: Post in [GitHub Discussions](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions)

### How do I use placeholders in playbooks?

Replace placeholders like `<instance-id>` or `<pod-name>` with your actual resource identifiers:

**Example:**

```
Playbook says: kubectl get pod <pod-name> -n <namespace>
You type: kubectl get pod my-app-pod-123 -n production
```

### Should I follow playbook steps in order?

Yes, generally follow steps sequentially. Steps are ordered from most common to specific causes. However, if you have strong evidence pointing to a specific step, you can jump ahead.

### What is the "Diagnosis" section for?

The Diagnosis section helps you correlate events with failures using time-based analysis. It's useful for:

- Finding root causes
- Identifying when issues started
- Correlating configuration changes with failures

## AWS Playbooks

### Do I need AWS credentials to use AWS playbooks?

Yes, you need appropriate AWS credentials and permissions to execute the diagnostic steps in AWS playbooks.

### Which AWS services are covered?

AWS playbooks are organized into 8 categories covering:

- **Compute**: EC2, Lambda, ECS, EKS, Fargate, Auto Scaling
- **Database**: RDS, DynamoDB
- **Storage**: S3
- **Networking**: VPC, ELB, Route 53, API Gateway, CloudFront
- **Security**: IAM, KMS, GuardDuty, WAF, Shield, Cognito
- **Monitoring**: CloudWatch, CloudTrail, Config, X-Ray
- **CI/CD**: CodePipeline, CodeBuild, CloudFormation
- **Proactive**: Capacity planning, cost optimization, compliance

### Can I use these playbooks in any AWS region?

Yes, but remember to replace `<region>` placeholders with your actual AWS region (e.g., `us-east-1`, `eu-west-1`).

## Kubernetes Playbooks

### Do I need kubectl access to use K8s playbooks?

Yes, you need `kubectl` configured with access to your Kubernetes cluster.

### How are Kubernetes playbooks organized?

K8s playbooks are organized into 13 numbered folders:

- `01-Control-Plane/` - Control plane issues
- `02-Nodes/` - Node problems
- `03-Pods/` - Pod issues (most common)
- `04-Workloads/` - Deployments, StatefulSets, etc.
- `05-Networking/` - Services, Ingress, DNS
- `06-Storage/` - Volumes, PVCs
- `07-RBAC/` - Permissions
- `08-Configuration/` - ConfigMaps, Secrets
- `09-Resource-Management/` - Quotas, limits
- `10-Monitoring-Autoscaling/` - Metrics, HPA
- `11-Installation-Setup/` - Installation issues
- `12-Namespaces/` - Namespace management
- `13-Proactive/` - Proactive monitoring and compliance

### What if my pod is in CrashLoopBackOff?

Start with `03-Pods/CrashLoopBackOff-pod.md`. This is one of the most common issues and has a comprehensive troubleshooting guide.

### How do I know which category my issue belongs to?

- **Pod not starting?** → `03-Pods/`
- **Service not accessible?** → `05-Networking/`
- **Permission denied?** → `07-RBAC/`
- **Volume mount failed?** → `06-Storage/`
- **Deployment not scaling?** → `04-Workloads/`

Each category folder has a README explaining what it covers.

## Technical Questions

### What is MTTR and how do playbooks help?

**MTTR (Mean Time To Recovery)** is the average time to restore a service after an incident. Playbooks help reduce MTTR by providing systematic troubleshooting steps, reducing guesswork and time spent searching for solutions.

### What is correlation analysis?

Correlation analysis helps you find relationships between events (like configuration changes) and symptoms (like service failures) by comparing timestamps. The Diagnosis section in each playbook guides you through this process.

### Can I customize these playbooks for my organization?

Yes! Since they're open-source, you can:

- Fork the repository
- Modify playbooks for your specific environment
- Add organization-specific steps
- Create internal versions

### Do these playbooks work with managed Kubernetes services?

Yes! These playbooks work with:

- **AWS EKS** (Elastic Kubernetes Service)
- **GKE** (Google Kubernetes Engine)
- **AKS** (Azure Kubernetes Service)
- **Self-managed clusters**

Some steps may vary slightly for managed services, but the core troubleshooting approach remains the same.

## Contributing

### How do I report a bug in a playbook?

1. Go to [GitHub Issues](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=bug_report.md) 2. Use the bug report template 3. Include the playbook name and what's wrong 4. Tag with appropriate labels

### How do I suggest a new playbook?

1. Check if a similar playbook exists 2. Create a [feature request](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues/new?template=feature_request.md) 3. Describe the issue and why a playbook would help 4. Optionally, create the playbook yourself!

### What makes a good playbook?

A good playbook:

- Follows the standard structure (Title, Meaning, Impact, Playbook, Diagnosis)
- Has 8-10 actionable diagnostic steps
- Uses placeholders for resource identifiers
- Includes correlation analysis in Diagnosis section
- Is clear and easy to follow

See [CONTRIBUTING.md](CONTRIBUTING.md) for detailed guidelines.

## Support

### Where can I get help?

- **GitHub Discussions**: [Ask questions](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions)
- **GitHub Issues**: [Report problems](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues)
- **Slack**: [Join our community](https://scoutflo.slack.com)
- **Documentation**: Check the README files in each folder

### How quickly will I get a response?

We aim to respond within:

- **Critical Issues**: 24 hours
- **Bug Reports**: 48 hours
- **Feature Requests**: 1 week
- **Questions**: 2-3 business days

### Can I use these playbooks in production?

Yes, but always:

- Test in non-production first
- Review steps before executing
- Understand what each command does
- Have a rollback plan
- Follow your organization's change management process

## Best Practices

### Should I bookmark specific playbooks?

Yes! Bookmark playbooks for issues you encounter frequently. You can also:

- Clone the repository locally
- Add to your team's runbook collection
- Integrate into your incident response tools

### How often are playbooks updated?

Playbooks are updated:

- When bugs are reported and fixed
- When new best practices emerge
- When community contributions are merged
- Continuously as the project evolves

### Can I share these playbooks with my team?

Absolutely! These are open-source and designed to be shared. You can:

- Share the repository link
- Print specific playbooks
- Integrate into your documentation
- Use in training sessions

---

**Still have questions?**

- [Open a Discussion](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/discussions)
- [Create an Issue](https://github.com/Scoutflo/scoutflo-SRE-Playbooks/issues)
- [Join Slack](https://scoutflo.slack.com)

# AWS Playbooks (Samples)

# Lambda Cold Start Delays Performance

## Meaning

Lambda cold start delays cause performance degradation (triggering latency alarms like LambdaDuration or LambdaColdStart) because function initialization takes too long, package size is large increasing startup time, runtime initialization is slow, provisioned concurrency is not configured, function code has heavy initialization logic, or Lambda VPC configuration adds latency. Lambda function invocations experience high latency, cold start delays increase response times, and first requests after idle periods are slow. This affects the serverless compute layer and impacts application performance, typically caused by initialization overhead, package size, or missing concurrency configuration; if using Lambda container images, cold start times may be longer and applications may experience inconsistent performance.

## Impact

Lambda function invocations experience high latency; cold start delays increase response times; user-facing latency degrades; first request after idle period is slow; Lambda function performance is inconsistent; LambdaDuration or LambdaColdStart alarms fire; application user experience is impacted; function startup time exceeds acceptable thresholds. Lambda function performance is inconsistent between cold and warm invocations; if using Lambda container images, cold starts may take significantly longer; applications may experience errors or performance degradation due to inconsistent latency; user-facing services experience spikes in response times.

## Playbook

1. Verify Lambda function `<function-name>` exists and AWS service health for Lambda in region `<region>` is normal. 2. Retrieve CloudWatch metrics for Lambda function `<function-name>` including Duration, InitDuration, and ColdStartDuration over the last 1 hour to identify cold start patterns, analyzing cold start frequency and duration. 3. Retrieve the Lambda Function `<function-name>` in region `<region>` and inspect its package size, runtime configuration, provisioned concurrency settings, memory allocation, and deployment package type (ZIP vs container image). 4. Query CloudWatch Logs for log group `/aws/lambda/<function-name>` and filter for initialization

logs, cold start indicators, or startup time patterns, including InitDuration metrics. 5. Retrieve CloudWatch alarms associated with Lambda function `<function-name>` with metric Duration and check for alarms in ALARM state related to latency, verifying alarm threshold configurations. 6. Retrieve CloudWatch metrics for Lambda function `<function-name>` including Invocations and ConcurrentExecutions to identify invocation frequency and analyze invocation patterns to identify idle periods causing cold starts. 7. Retrieve the Lambda Function `<function-name>` VPC configuration and verify if function is configured in VPC, checking if VPC configuration adds initialization latency. 8. Retrieve the Lambda Function `<function-name>` deployment package type and verify

## 01-Compute/Connection-Timeout-SSH-Issues-EC2.md

# EC2 Instance Connection Timeout (SSH Issues)

## Meaning

EC2 instance SSH connections timeout or fail (triggering alarms like EC2InstanceStatusCheckFailed or connection timeout errors) because security group rules block SSH access, the instance lacks a public IP address, key pair mismatches occur, network connectivity issues prevent SSH access, or Systems Manager Session Manager is not configured. Users experience "Connection timed out" or "Connection refused" errors when attempting SSH access, instances show "running" state but status checks may indicate failures in AWS console, and security group rules may block port 22 access. This affects the compute layer and prevents remote access, typically caused by security group restrictions, missing public IP addresses, network connectivity issues, or IMDSv2 enforcement blocking access; if instances host container workloads, pod troubleshooting may be blocked and container orchestration tasks may fail.

## Impact

EC2InstanceStatusCheckFailed alarms may fire; administrators cannot access instances; SSH connection timeout errors occur; "Connection timed out" or "Connection refused" messages appear in logs; instance remains inaccessible; remote management fails; troubleshooting is blocked. Instance shows "running" state in AWS console but status checks indicate failures; manual

intervention required; operational tasks delayed; users cannot perform administrative actions; configuration changes cannot be applied. Instance health appears degraded in CloudWatch dashboards; Auto Scaling may mark instance as unhealthy; if instances host container workloads, pod scheduling may fail, containers may be unable to start, or cluster connectivity issues may occur; container orchestration tasks may show failures; applications may experience errors or performance degradation.

## Playbook

### For AI Agents (NLP)

1. Verify instance `<instance-id>` is in "running" state and AWS service health for EC2 in region `<region>` is normal. 2. Retrieve the Security Group `<security-group-id>` associated with EC2 instance `<instance-id>` and inspect inbound rules for SSH port 22 access, verifying source IP addresses or CIDR blocks. 3. Retrieve the EC2 Instance `<instance-id>` in region `<region>` and verify public IP address or Elastic IP assignment, checking if instance is in a public subnet with internet gateway route. 4. Verify the key pair `<key-pair-name>` matches the key pair assigned to instance `<instance-id>` by retrieving instance key pair configuration. 5. Retrieve the EC2 Instance `<instance-id>` metadata service configuration and verify IMDSv2 enforcement settings. 6. Retrieve the EC2 Instance `<instance-id>` IAM role configuration and verify instance profile is attached, checking IAM role association. 7. Retrieve CloudWatch Logs for EC2 serial console output for instance `<instance-id>` and filter for connection errors or authentication failures. 8. Retrieve the Route Table `<route-table-id>` and Network ACL `<n

## 01-Compute/Connection-Timeout-from-Lambda-RDS.md

# RDS Connection Timeout from Lambda

## Meaning

Lambda functions cannot connect to RDS databases (triggering connection timeout errors or LambdaRDSConnectionTimeout alarms) because database is not running, VPC security groups block inbound access, subnet group configuration is incorrect, database is in wrong availability zone, public access is disabled when needed, Lambda VPC configuration is missing, connectivity tests from same VPC fail, or Lambda ENI creation delays prevent connections. Lambda functions

cannot access database, connection timeout errors occur, and CloudWatch Logs show connection failures. This affects the serverless and database integration layer and blocks data access, typically caused by VPC networking issues, security group restrictions, or Lambda VPC configuration problems; if using Lambda container images, VPC configuration may differ and applications may experience database connection errors.

## Impact

Lambda functions cannot access database; connection timeout errors occur; database queries fail; application data operations error; Lambda function execution fails; connection pool exhaustion may occur; RDS connection timeout alarms may fire; service-to-database communication breaks; data processing workflows fail. LambdaRDSConnectionTimeout alarms fire; if using Lambda container images, VPC ENI creation may take longer; applications may experience errors or performance degradation due to database unavailability; Lambda function cold starts with VPC may add significant latency.

## Playbook

1. Verify Lambda function `<function-name>` and RDS instance `<rds-instance-id>` exist, and AWS service health for Lambda and RDS in region `<region>` is normal. 2. Retrieve the RDS Instance `<rds-instance-id>` in region `<region>` and confirm the database is running by checking instance status, verifying instance is in "available" state. 3. Retrieve the Security Group `<security-group-id>` associated with RDS instance `<rds-instance-id>` and check VPC security groups ensuring inbound rules allow access from Lambda execution environment, verifying Lambda security group source. 4. Retrieve the Lambda Function `<function-name>` VPC configuration and verify Lambda is configured in VPC with correct subnets and security groups, verify Lambda security group has outbound rules allowing RDS access, and verify Lambda subnets are in same VPC as RDS subnet group, checking VPC subnet configuration, security group egress rules, and VPC ID match. 5. Retrieve the RDS Instance `<rds-instance-id>` subnet group configuration and verify subnet group and whether the database is in the correct availability zone, checking subnet group availability zones. 6. Query CloudWatch Logs for log groups containing VPC Flow Logs or Lambda function logs and filter for blocked traffic from Lambda security group to RDS endpoint `<rds-endpoint>` or VPC ENI creation errors, including connection timeout errors, checking flow log and Lambda log analysis. 7. Retrieve Cloud

## 01-Compute/Exceeds-Memory-Limit-Lambda.md

# Lambda Function Exceeds Memory Limit

## Meaning

A Lambda function exceeds its memory limit (triggering out-of-memory errors or LambdaMemoryError alarms) because function memory allocation is insufficient for workload, memory leaks cause gradual memory growth, function processes large datasets exceeding memory, concurrent executions exhaust available memory, or Lambda function memory configuration is too low. Lambda function executions fail with out-of-memory errors, function invocations are terminated, and CloudWatch metrics show memory utilization exceeding limits. This affects the serverless compute layer and blocks function execution, typically caused by memory allocation issues, memory leaks, or data processing requirements; if using Lambda container images, memory behavior may differ and applications may experience function execution failures.

## Impact

Lambda function executions fail; out-of-memory errors occur; function invocations are terminated; application workflows break; Lambda function errors increase; LambdaMemoryError alarms fire; retry attempts fail; function cannot complete processing; service reliability degrades. Lambda function memory errors appear in CloudWatch Logs; if using Lambda container images, memory allocation behavior may differ; applications may experience errors or performance degradation due to incomplete function execution; downstream services may not receive expected data.

## Playbook

1. Verify Lambda function `<function-name>` exists and AWS service health for Lambda in region `<region>` is normal. 2. Retrieve CloudWatch metrics for Lambda function `<function-name>` including MemoryUtilization, Duration, and Errors over the last 1 hour to identify memory usage patterns, analyzing memory utilization trends. 3. Retrieve the Lambda Function `<function-name>` in region `<region>` and inspect its memory configuration, timeout settings, function code size, and deployment package type (ZIP vs container image). 4. Query CloudWatch Logs for log group `/aws/lambda/<function-name>` and filter for out-of-memory error patterns, memory-related exceptions, or function termination errors, including memory utilization logs. 5. Retrieve CloudWatch alarms associated with Lambda function `<function-name>` with metric Errors or MemoryUtilization and check for alarms in ALARM state, verifying alarm threshold configurations.

6. Retrieve CloudWatch metrics for Lambda function `<function-name>` including ConcurrentExecutions and verify if concurrent executions contribute to memory exhaustion, analyzing concurrency patterns. 7. Retrieve the Lambda Function `<function-name>` reserved concurrency settings and verify reserved concurrency configuration, checking if concurrency limits affect memory availability. 8. Query CloudWatch Logs for log group `/aws/lambda/<function-name>` and filter for memory allocation errors or memory-related exception patterns, checking for memory leak indicators. 9. Compare CloudWatch metrics for Lambda

## 01-Compute/Failing-on-EC2-Instances-CodeDeploy.md

# CodeDeploy Failing on EC2 Instances

## Meaning

CodeDeploy deployments fail on EC2 instances (triggering deployment failures or CodeDeployDeploymentFailure alarms) because CodeDeploy agent is not running, IAM instance profile permissions are insufficient, deployment group configuration is incorrect, application stop scripts fail, instance health checks fail during deployment, or CodeDeploy agent version is incompatible. CodeDeploy deployments fail, application updates cannot be deployed, and EC2 instances do not receive new code. This affects the CI/CD and deployment layer and blocks application updates, typically caused by agent issues, permission problems, or configuration errors; if instances host container workloads, CodeDeploy behavior may differ and applications may experience deployment failures.

## Impact

CodeDeploy deployments fail; application updates cannot be deployed; EC2 instances do not receive new code; deployment automation is blocked; instance health checks fail; deployment rollbacks occur; application versions remain outdated; deployment processes are interrupted. CodeDeployDeploymentFailure alarms may fire; if instances host container workloads, CodeDeploy behavior may differ; applications may experience errors or performance degradation due to failed deployments; application versions may remain outdated.

## Playbook

1. Verify CodeDeploy application `<application-name>` and deployment group `<deployment-group-name>` exist, and AWS service health for CodeDeploy in region `<region>` is normal. 2. Retrieve CodeDeploy deployment history for deployment group `<deployment-group-name>` and inspect recent deployment failures, deployment status, and instance deployment states, analyzing failure patterns. 3. Query CloudWatch Logs for log groups containing CodeDeploy agent logs and filter for deployment failure patterns, agent errors, or health check failures, including agent error messages. 4. Retrieve the CodeDeploy Application `<application-name>` and Deployment Group `<deployment-group-name>` in region `<region>` and inspect deployment configuration, health check settings, and instance selection criteria, verifying deployment group configuration. 5. Retrieve the IAM instance profile `<instance-profile-name>` attached to EC2 instances in deployment group `<deployment-group-name>` and inspect its policy permissions for CodeDeploy operations, verifying IAM permissions. 6. List EC2 instances in deployment group `<deployment-group-name>` and check CodeDeploy agent status, instance health, and deployment state, verifying agent is running. 7. Retrieve the CodeDeploy Deployment Group `<deployment-group-name>` deployment configuration and verify deployment configuration settings, checking deployment type and deployment settings. 8. Query CloudWatch Logs for log groups containing CloudTrail events and filter for CodeDeploy deployment group or application modification events related to `<deployment-group-name>` within the last 24 hours,

## 01-Compute/High-CPU-Utilization-EC2.md

# EC2 Instance High CPU Utilization

## Meaning

An EC2 instance experiences sustained high CPU utilization (triggering alarms like CPUUtilizationHigh) because CPU-intensive processes consume excessive resources, application code inefficiencies cause high CPU usage, instance type is undersized for workload, background processes compete for CPU resources, or CloudWatch metrics indicate CPU utilization consistently above thresholds. CPU utilization metrics exceed 80-90% for extended periods, CloudWatch alarms trigger for high CPU, and application performance degrades. This affects the compute layer and impacts service performance, typically caused by resource constraints, application inefficiencies, or instance sizing issues; if instances host container workloads, high CPU may affect container scheduling and applications may experience performance degradation.

## Impact

CPUUtilizationHigh alarms fire; application performance degrades; response times increase; user-facing services become slow; instance becomes unresponsive; Auto Scaling may trigger unnecessary scaling; other processes on the instance are starved of CPU resources; system stability is compromised. Instance CPU utilization consistently exceeds 80-90%; if instances host container workloads, container scheduling may be affected and applications may experience performance degradation; application workflows may slow down or fail; user-facing services experience increased latency.

## Playbook

1. Verify instance `<instance-id>` exists and is in "running" state, and AWS service health for EC2 in region `<region>` is normal. 2. Retrieve CloudWatch metrics for EC2 instance `<instance-id>` including CPUUtilization over the last 1 hour to identify CPU usage patterns and spikes, analyzing CPU trend over time. 3. Retrieve the EC2 Instance `<instance-id>` in region `<region>` and inspect its instance type configuration, verifying instance type is appropriate for workload. 4. Query CloudWatch Logs for log groups containing application logs for instance `<instance-id>` and filter for error patterns or performance-related log entries indicating CPU-intensive operations, including process-level CPU usage indicators. 5. Retrieve CloudWatch alarms associated with instance `<instance-id>` with metric CPUUtilization and check for alarms in ALARM state, verifying alarm threshold configurations. 6. List EC2 instances in region `<region>` with the same instance type as `<instance-id>` and compare CPU utilization patterns to determine if the issue is instance-specific, analyzing CPU utilization across similar instances. 7. Retrieve the Auto Scaling Group `<asg-name>` associated with instance `<instance-id>` and inspect scaling policies and target tracking configurations to verify if high CPU triggers scaling actions, checking scaling policy CPU thresholds. 8. Retrieve CloudWatch metrics for EC2 instance `<instance-id>` including NetworkIn, NetworkOut, and DiskReadOps to identify if high CPU correlates with

## 01-Compute/IAM-Role-Not-Attaching-to-Service-Account-EKS.md

# EKS IAM Role Not Attaching to Service

# Account

## Meaning

EKS IAM role is not attaching to service account (triggering permission failures or EKSServiceAccountIRSAFailure alarms) because IAM role trust policy does not allow service account, service account annotation is incorrect, IAM role ARN is wrong, OIDC provider is not configured, service account and IAM role are in different accounts, or EKS cluster OIDC provider is missing. EKS service account cannot assume IAM role, pod permissions are insufficient, and application pods cannot access AWS services. This affects the container orchestration and security layer and blocks service access, typically caused by IRSA (IAM Roles for Service Accounts) configuration issues, OIDC provider problems, or trust policy errors; if using EKS with multiple namespaces, service account configuration may differ and applications may experience permission failures.

## Impact

EKS service account cannot assume IAM role; pod permissions are insufficient; service account IAM integration fails; pod IAM role attachment errors occur; application pods cannot access AWS services; IAM role-based access fails; Kubernetes service account automation is ineffective. EKSServiceAccountIRSAFailure alarms may fire; if using EKS with multiple namespaces, service account configuration may differ; applications may experience errors or performance degradation due to missing AWS service permissions; pods cannot access required AWS resources.

## Playbook

1. Verify EKS cluster `<cluster-name>` exists and AWS service health for EKS and IAM in region `<region>` is normal. 2. Retrieve the EKS Cluster `<cluster-name>` in region `<region>` and inspect its OIDC provider configuration, OIDC provider URL, and IAM role trust relationships, verifying OIDC provider is configured. 3. Query CloudWatch Logs for log groups containing EKS pod logs and filter for IAM role attachment failure patterns, permission errors, or service account annotation errors, including pod error messages. 4. Retrieve the IAM role `<role-arn>` that should attach to service account and inspect its trust policy, service account trust relationships, and OIDC provider configuration, verifying trust policy format. 5. List EKS service accounts in cluster `<cluster-name>` and check service account annotations, IAM role annotations, and service account configurations, verifying annotation format. 6. Query CloudWatch Logs for log groups containing CloudTrail events

and filter for EKS service account IAM role assumption failures or permission denied events, including AssumeRole failures. 7. Retrieve the EKS Cluster `<cluster-name>` OIDC provider issuer URL and verify OIDC provider is accessible, checking OIDC provider status. 8. Retrieve the IAM role `<role-arn>` trust policy and verify trust policy condition matches service account namespace and name, checking trust policy condition format. 9. Query CloudWatch Logs for log groups containing CloudTrail events and filter for EKS c

## 01-Compute/Image-Pull-Failing-in-ECS-Docker.md

# Docker Image Pull Failing in ECS

## Meaning

Docker image pull is failing in ECS (triggering image pull errors or ECSImagePullFailed alarms) because ECR repository access is denied, IAM task execution role lacks ECR permissions, image does not exist in repository, image tag is incorrect, ECR repository policy restricts access, or ECS task network configuration prevents ECR access. ECS tasks cannot pull Docker images, container image pull errors occur, and task startup fails. This affects the container orchestration layer and blocks container deployment, typically caused by ECR permission issues, image availability problems, or network configuration errors; if using ECS with Fargate, ECR access behavior may differ and applications may experience image pull failures.

## Impact

ECS tasks cannot pull Docker images; container image pull errors occur; task startup fails; ECR image access is denied; container deployment is blocked; image pull automation fails; task definition cannot launch containers; application containers cannot start. ECSImagePullFailed alarms may fire; if using ECS with Fargate, ECR access behavior may differ; applications may experience errors or performance degradation due to failed container startup; container workloads cannot be deployed.

## Playbook

1. Verify ECS task definition `<task-definition-arn>` exists and AWS service health for ECS and ECR in region `<region>` is normal. 2. Retrieve the ECS Task Definition `<task-definition-arn>`

and inspect its container image configurations, image URIs, and ECR repository references, verifying image URI format. 3. Retrieve the IAM role `<role-name>` used for ECS task execution and inspect its policy permissions for ECR operations including GetAuthorizationToken, BatchGetImage, and GetDownloadUrlForLayer, verifying IAM permissions. 4. Query CloudWatch Logs for log groups containing ECS task logs and filter for image pull failure patterns, ECR access denied errors, or Docker pull errors, including error message details. 5. Retrieve the ECR Repository `<repository-name>` referenced in task definition and inspect its repository policy, image tags, and repository access settings, verifying image exists. 6. List ECS task failures in cluster `<cluster-name>` and check for image pull error patterns and task failure reasons, analyzing failure patterns. 7. Retrieve the ECS Task Definition `<task-definition-arn>` network configuration if using VPC and verify network connectivity to ECR, checking if VPC configuration blocks ECR access. 8. Retrieve CloudWatch metrics for ECR repository `<repository-name>` including ImagePullCount if available and verify repository activity, checking if repository is accessible. 9. Query CloudWatch Logs for log groups containing CloudTrail events and filter for ECR repository policy or IAM role policy modification events related to task execution role `<role-name>` within the last 24 hours, checking for permission changes.

### Diagnosis

1. Analyze CloudWatc

# K8s Playbooks (Samples)

## 01-Control-Plane/APIServerHighLatency-control-plane.md

--- title: API Server High Latency - Control Plane weight: 263 categories: - kubernetes - control-plane ---

# APIServerHighLatency-control-plane

### Meaning

The Kubernetes API server is experiencing high latency (triggering KubeAPILatencyHigh alerts) because it is under heavy load, experiencing resource constraints, network issues, or storage backend performance problems. API server metrics show high request latency exceeding 1 second, API server request queue depth metrics indicate request backlog, and etcd performance metrics may show latency issues. This affects the control plane and indicates API server performance degradation that delays cluster operations, typically caused by heavy load, resource constraints, etcd performance issues, or admission webhook timeouts; applications using Kubernetes API may show errors.

## Impact

API requests take longer than 1 second to complete; kubectl commands experience delays; controller reconciliation is slow; deployments and updates are delayed; cluster operations are sluggish; timeouts may occur; KubeAPILatencyHigh alerts fire; API server metrics show high request latency; cluster responsiveness is degraded. API server metrics show sustained high request latency; API server request queue depth metrics indicate request backlog; etcd performance metrics may show latency issues; applications using Kubernetes API may experience errors or performance degradation; controller reconciliation is slow.

## Playbook

1. Describe API server pods in namespace kube-system with label component=kube-apiserver to retrieve detailed API server pod information including status, restarts, resource usage, and any warning conditions.

2. Retrieve events in namespace kube-system sorted by timestamp, filtering for API server errors, throttling events, or performance-related issues.

3. Retrieve API server pod metrics and logs to identify high latency patterns, focusing on request duration (apiserver_request_duration_seconds), queue depth (apiserver_request_queue_depth), and error rates.

4. Check etcd leader status by retrieving the etcd endpoints in kube-system namespace and verify if etcd leader election issues are occurring.

5. Check etcd performance metrics and health status since API server depends on etcd for storage backend, focusing on etcd_request_duration_seconds and etcd_leader_changes metrics to verify if etcd latency is contributing to API server delays.

6. Check API server admission webhook latency by reviewing admission webhook metrics (apiserver_admission_webhook_admission_duration_seconds) to verify if webhook timeouts are causing delays.

7. Check API server pod resource usage (CPU and memory) to verify if resource constraints are causing performance degradation.

8. Review API server audit logs or metrics for patterns in request types, clients, or operations that may be causing high load.

## Diagnosis

1. Analyze API server pod events from Playbook to identify if API server is restarting, resource c

## 01-Control-Plane/CannotAccessAPI-control-plane.md

--- title: Cannot Access API - Control Plane weight: 255 categories: - kubernetes - control-plane ---

# CannotAccessAPI-control-plane

## Meaning

External clients such as kubectl, CI/CD systems, or external controllers cannot establish TCP/TLS connections to the Kubernetes API server endpoint (potentially triggering KubeAPIDown alerts), indicating a control-plane reachability or exposure problem. API server connectivity failures prevent external access to the cluster control plane.

## Impact

All kubectl commands fail; cluster management operations cannot be performed; controllers cannot reconcile state; cluster becomes unmanageable; applications may continue running but cannot be updated or scaled; KubeAPIDown alerts may fire; API server unreachable from external networks; connection timeouts or connection refused errors occur; cluster management tools fail.

## Playbook

1. Describe API server pods in namespace kube-system with label component=kube-apiserver to retrieve detailed API server pod information including status, restarts, conditions, and any error messages.

2. Retrieve events in namespace kube-system sorted by timestamp, filtering for API server connectivity errors or failures.

3. Retrieve cluster information including the API server endpoint and record the exact URL and IP/port being used for API access.

4. From a pod inside the cluster, verify TCP/TLS connectivity to the API server endpoint to test basic connectivity.

5. On a control plane node, verify that the API server is listening on the expected port and interface to confirm the API server process is bound correctly.

6. Check firewall and security group rules on the control plane node and surrounding network (cloud firewall, security groups, NACLs) for rules that might block ingress or egress on port 6443.

7. List NetworkPolicy resources in kube-system and other relevant namespaces and inspect whether any policies restrict traffic from external clients or ingress gateways to the API server.

## Diagnosis

1. Analyze API server pod events from Playbook to identify if API server is running and healthy. If events show pod failures, restarts, or unhealthy status, API server unavailability is the root cause rather than external access issues.

2. If events indicate API server is healthy, verify network connectivity from Playbook steps 4-5. If API server is running but external access fails, network path issues are the likely cause.

3. If events indicate NetworkPolicy changes, examine policy modifications from Playbook step 7. If NetworkPolicy events occurred before access failures began, policy changes may have blocked external client access.

4. If events indicate firewall or security group changes, correlate infrastructure change timestamps with failure onset. If firewall modifications occurred at timestamps preceding access failures, network security changes blocked connectivity.

5. If events indicate load balancer issues, verify load balancer status and configura

## 01-Control-Plane/CertManagerACMEOrderFailed-cert.md

--- title: Cert Manager ACME Order Failed weight: 43 categories: [kubernetes, cert-manager] ---

# CertManagerACMEOrderFailed

## Meaning

ACME certificate order has failed (triggering CertManagerACMEOrderFailed alerts) because the ACME challenge (HTTP-01 or DNS-01) could not be completed successfully. Order resource shows Failed state, domain validation did not pass, and the certificate cannot be issued. This affects certificates using ACME issuers like Let's Encrypt; new certificates fail to issue; renewals fail; HTTPS may be unavailable.

## Impact

CertManagerACMEOrderFailed alerts fire; certificate cannot be issued; HTTPS endpoints have no valid certificate; new deployments requiring TLS are blocked; certificate renewals fail; browsers show security errors; API clients reject connections; ingress cannot serve HTTPS traffic securely.

## Playbook

1. Retrieve the Order resource associated with the failing Certificate and check its status and failure reason.

2. Retrieve Challenge resources under the Order and identify which domains failed validation.

3. For HTTP-01 challenges: verify the challenge path is accessible from the internet at http://domain/.well-known/acme-challenge/<token>.

4. For DNS-01 challenges: verify the _acme-challenge TXT record exists and has the correct value.

5. Check ingress configuration to ensure challenge paths are not blocked or redirected.

6. Verify firewall and network policies allow incoming HTTP(S) for validation.

7. Check ACME server response in cert-manager logs for specific error details.

## Diagnosis

For HTTP-01 challenges, verify that the challenge URL returns the expected token value from external internet locations, using curl or HTTP testing tools as supporting evidence.

Check if ingress controllers have configurations that interfere with ACME challenges (forced HTTPS redirects, authentication requirements), using ingress annotations and routing rules as supporting evidence.

For DNS-01 challenges, verify DNS propagation is complete and TXT record is resolvable from multiple locations, using DNS query tools and propagation checkers as supporting evidence.

Check for Let's Encrypt rate limits by examining error messages mentioning rate limiting or too many requests, using ACME server responses in logs as supporting evidence.

Verify domain ownership and DNS configuration points to the correct ingress, using DNS records and domain registration as supporting evidence.

If no correlation is found within the specified time windows: use ACME staging server for testing to avoid rate limits, verify DNS credentials for DNS-01, check for CAA records blocking issuance, temporarily disable HTTPS redirects during challenge, contact ACME provider if persistent failures.

## 01-Control-Plane/CertManagerControllerHighError-cert.md

--- title: Cert Manager Controller High Error Rate weight: 44 categories: [kubernetes, cert-manager] ---

# CertManagerControllerHighError

## Meaning

Cert-manager controller is experiencing high error rate (triggering CertManagerControllerHighError alerts) because certificate operations are failing frequently, indicating systemic issues with issuers, configuration, or cluster connectivity. Controller metrics show elevated error counts, certificate operations are not completing successfully, and certificate management is degraded. This affects certificate issuance and renewal across the cluster; certificates may not be issued or renewed; increased manual intervention needed.

## Impact

CertManagerControllerHighError alerts fire; certificate operations fail frequently; new certificate requests may fail; renewals may be delayed; certificate status is unreliable; manual intervention required more often; certificate-dependent deployments are delayed; security posture may be affected by renewal failures.

## Playbook

1. Retrieve cert-manager controller logs and filter for ERROR level messages to identify common failure patterns.

2. Check controller metrics for error rate breakdown by operation type (issuance, renewal, sync).

3. Identify which Issuers or ClusterIssuers have the most failures.

4. Verify API server connectivity and RBAC permissions for cert-manager.

5. Check for Certificate or CertificateRequest resources stuck in error states.

6. Verify webhook service is healthy and responding correctly.

7. Check for resource contention or throttling affecting controller operation.

## Diagnosis

Categorize errors by type and identify if errors are concentrated on specific issuers, namespaces, or certificate types, using error logs and metrics breakdown as supporting evidence.

Check for API server connectivity issues that prevent cert-manager from reading or updating resources, using API server latency metrics and error logs as supporting evidence.

Verify issuer credentials (ACME accounts, CA secrets, Vault tokens) are valid and not expired, using issuer status and credential validation as supporting evidence.

Check for rate limiting from external ACME providers causing operation failures, using ACME responses in logs as supporting evidence.

Analyze controller restart history and verify if restarts correlate with error spikes (suggesting controller instability), using pod restart timestamps and error metrics as supporting evidence.

If no correlation is found within the specified time windows: review recent cert-manager or cluster upgrades, verify CRD compatibility, check for memory or CPU constraints, review RBAC changes, consider cert-manager reinstallation if controller is corrupt.

## 01-Control-Plane/CertManagerDown-cert.md

--- title: Cert Manager Down weight: 42 categories: [kubernetes, cert-manager] ---

# CertManagerDown

## Meaning

Cert-manager controller is not running or not healthy (triggering CertManagerDown alerts) because the cert-manager pods are crashing, not scheduled, or not responding to health checks. Cert-manager deployment shows zero ready replicas, no certificate operations are being processed, and new certificate issuance and renewals are blocked. This affects all certificate management in the cluster; new certificates cannot be issued; renewals will not occur; certificate infrastructure is non-functional.

## Impact

CertManagerDown alerts fire; no new certificates can be issued; certificate renewals stop; pending certificate requests are not processed; new deployments requiring certificates are blocked; expiring certificates are not renewed; HTTPS outages will occur as certificates expire; compliance requirements are at risk; security posture degrades.

## Playbook

1. Retrieve cert-manager deployments in the cert-manager namespace and verify replica status.

2. Check cert-manager controller pod status, restart count, and events.

3. Retrieve cert-manager controller logs for startup failures or runtime errors.

4. Verify cert-manager webhook deployment is healthy (required for cert-manager operation).

5. Check for resource constraints preventing pod scheduling (CPU, memory, node availability).

6. Verify cert-manager CRDs are installed correctly and not corrupted.

7. Check for conflicts with other operators or admission webhooks.

## Diagnosis

Analyze controller pod status and identify if pods are CrashLooping, Pending, or missing, using pod status and events as supporting evidence.

Check for OOMKilled events indicating memory limits are too low for cert-manager operation, using container status and memory metrics as supporting evidence.

Verify webhook service is accessible as cert-manager cannot function without its webhook, using webhook pod status and service endpoints as supporting evidence.

Check for CRD version mismatches after cert-manager upgrades that may cause controller failures, using CRD versions and controller compatibility as supporting evidence.

Analyze controller logs for specific error patterns (API access denied, leader election failures, watch errors), using log analysis as supporting evidence.

If no correlation is found within the specified time windows: restart cert-manager pods, verify RBAC permissions are correct, check for API server connectivity, reinstall cert-manager if CRDs are corrupted, verify leader election lease is not stuck.

## 01-Control-Plane/CertificateExpired-control-plane.md

--- title: Certificate Expired - Control Plane weight: 249 categories: - kubernetes - control-plane ---

# CertificateExpired-control-plane

## Meaning

One or more cluster X.509 certificates (for the API server, kubelets, or etcd) have passed their validity period, causing TLS handshakes and authenticated requests between core Kubernetes

components to start failing. Certificate expiration triggers x509 certificate errors, authentication failures, and component communication breakdowns.

## Impact

All API operations fail; cluster becomes completely non-functional; nodes cannot communicate with control plane; kubectl commands fail; controllers stop reconciling; cluster is effectively down; certificate expiration errors appear in logs; TLS handshake failures occur; KubeAPIDown or KubeletDown alerts may fire; authentication errors prevent cluster operations.

## Playbook

1. List API server pods in namespace kube-system with label component=kube-apiserver to check API server pod status and identify any pods experiencing certificate-related failures.

2. Retrieve events in namespace kube-system sorted by timestamp, filtering for certificate errors or TLS handshake failures.

3. On a control plane node, check certificate expiration dates for all control plane, kubelet, and etcd certificates.

4. Retrieve logs from the API server pod in kube-system (or system logs if running as a service) and filter for certificate errors such as x509: certificate has expired or failed TLS handshakes.

5. List all nodes and inspect their Ready status and conditions to identify nodes that have lost contact with the control plane around the time of certificate errors.

6. Check the status of any certificate rotation automation (such as cert-manager or external-secrets operators) by listing their pods and verifying they are running without errors.

7. Using the certificate inspection output and any stored CA files, verify that cluster CA certificates are still valid and have not reached or exceeded their expiration dates.

## Diagnosis

1. Analyze certificate-related events from Playbook to identify which certificates have expired and when expiration occurred. If events show x509 certificate errors or TLS handshake failures, use event timestamps to pinpoint when authentication failures began.

2. If events indicate certificate expiration, verify certificate expiration dates from Playbook step 3. If certificate expiration timestamps from `kubeadm certs check-expiration` align with authentication error event timestamps, certificate expiration is confirmed as root cause.

3. If events indicate control plane component restarts or failures, correlate component restart timestamps from events with certificate expiration times. If restarts began at or shortly after certificate expiration, expired certificates caused component failures.

4. If events indicate CSR approval failures or kubelet communication issues, check whether CSR-related events began failing around certifica

## 01-Control-Plane/CertificateExpiringCritical-cert.md

--- title: Certificate Expiring Critical weight: 45 categories: [kubernetes, cert-manager] ---

# CertificateExpiringCritical

## Meaning

Certificate is critically close to expiration (triggering CertificateExpiringCritical, CertManagerCertExpiryCritical alerts, typically 7 days or less before expiry) because automatic renewal has failed repeatedly and immediate action is required. Certificate will expire very soon causing service outage, and all renewal attempts have failed. This is a critical issue requiring immediate attention; HTTPS services will fail when the certificate expires; customer impact is imminent.

## Impact

CertificateExpiringCritical alerts fire; imminent service outage; HTTPS will fail within days; urgent action required; browsers will block access; API integrations will break; mobile apps will fail; compliance violations imminent; customer-facing outage is near-certain without intervention; business impact is significant.

## Playbook

1. Immediately check certificate expiration date and calculate exact time until expiry.

2. Retrieve Certificate resource status and identify all failure reasons in conditions.

3. Check all CertificateRequest resources for detailed failure information.

4. Verify Issuer or ClusterIssuer can issue certificates by testing with a new certificate.

5. Attempt manual certificate renewal by deleting the Certificate's secret to force re-issuance.

6. If ACME: verify all challenge requirements are met (DNS, HTTP connectivity).

7. Prepare fallback plan: obtain certificate manually from CA if automation cannot be fixed in time.

## Diagnosis

This is a critical alert - prioritize resolution over root cause analysis initially. Get the certificate renewed first, then investigate why automatic renewal failed.

Check if issuer credentials have expired (ACME account, CA credentials, Vault token) preventing any issuance, using issuer status and credential validation as supporting evidence.

Verify network connectivity for ACME challenges has not been disrupted by firewall or infrastructure changes, using connectivity tests and recent infrastructure changes as supporting evidence.

Check for Let's Encrypt rate limits that may be blocking issuance due to previous failed attempts, using ACME server responses and rate limit status as supporting evidence.

Verify DNS configuration for the domain is correct and resolvable, using DNS queries and propagation checks as supporting evidence.

Emergency actions if automated renewal cannot be fixed: manually obtain certificate from Let's Encrypt using certbot or other ACME client, manually obtain certificate from paid CA, temporarily use self-signed certificate with client-side trust (not recommended for public services).

## 01-Control-Plane/CertificateExpiringSoon-cert.md

--- title: Certificate Expiring Soon weight: 41 categories: [kubernetes, cert-manager] ---

# CertificateExpiringSoon

## Meaning

Certificate is approaching expiration (triggering CertificateExpiringSoon, CertManagerCertExpirySoon alerts, typically 30 days or less before expiry) because automatic renewal has not occurred and the certificate will expire soon. Certificate resource shows upcoming expiration date, renewal may be failing silently, and services will fail when the certificate expires. This affects all services using this certificate; proactive renewal is needed to prevent outage; HTTPS will fail after expiration.

## Impact

CertificateExpiringSoon alerts fire; warning of impending outage; if not resolved, HTTPS will fail at expiration; browsers will block access; API connections will fail; mobile apps will reject certificates; mTLS will break; ingress will serve invalid certificates; compliance violations will occur; customer-facing services will be impacted.

## Playbook

1. Retrieve the Certificate `<certificate-name>` in namespace `<namespace>` and verify expiration date and renewal status.

2. Check the renewBefore setting and calculate when cert-manager should have attempted renewal.

3. Retrieve CertificateRequest resources to see if renewal requests are being created and their status.

4. Verify the Issuer or ClusterIssuer is still functioning and can issue new certificates.

5. Check cert-manager controller logs for renewal attempt failures.

6. Verify DNS and ingress configuration still allows ACME challenges if using Let's Encrypt.

7. Manually trigger renewal by deleting the Certificate's Secret if automatic renewal is stuck.

## Diagnosis

Compare current time with renewBefore threshold and verify whether cert-manager should have renewed already, using certificate spec and current timestamp as supporting evidence.

Check for failed CertificateRequest resources that indicate renewal was attempted but failed, using CertificateRequest status and events as supporting evidence.

Verify Issuer configuration has not changed and credentials are still valid (API keys, service account permissions), using Issuer status and credential validity as supporting evidence.

Check for network changes that may block ACME validation (firewall rules, ingress changes, DNS changes), using connectivity tests and configuration history as supporting evidence.

Analyze cert-manager controller logs for any errors during renewal process, using log analysis and error patterns as supporting evidence.

If no correlation is found within the specified time windows: manually delete the secret to trigger re-issuance, verify issuer is working with a test certificate, check for cert-manager webhook issues, consider manually issuing certificate as temporary fix, escalate if issuer is permanently broken.

# Sentry Playbooks (Samples)

## 01-Error-Tracking/APICallFailed-Error-application.md

# APICallFailed-Error-application

## Meaning

External API call failure occurs (triggering Sentry error issue) because external API request fails, causing API operations to fail when attempting to communicate with external services. Error events display in Sentry Issue Details page with error message patterns "API call failed", "HTTP error", "request failed", stack traces show API client layer, and error levels indicate Error or Fatal severity. This affects the external API integration layer and indicates API integration issues typically caused by external service failures, HTTP errors, or API client configuration problems; API operations fail.

## Impact

Sentry error issue alerts fire; external API call failures occur; API operations fail; applications return API errors; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate API call failures; error events display in Sentry Issue Details page. External API integration breaks; affected user count grows; error count increases continuously; issues show High or Medium priority in Sentry dashboard; application functionality degrades; feature failures occur. API errors increase; external service dependencies fail.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify API call failure with "API call failed", "HTTP error", or "request failed" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected API client files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract API endpoint details from error message ( `metadata.value` ) or stack trace context including API URL and HTTP status code to identify target external API service and failure type.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze API integration or configuration changes.

6. Search ELK logs around the error timestamp for: - API request/response logs showing the full request details and response codes - External service connectivity errors or timeouts - Circuit breaker state changes (open/closed/half-open) if implemented - Rate limiting or throttling logs from the external API - DNS resolution or network connectivity issues

7. List similar issues for project `<project-name>` and check if they share common release or deployment time

## 01-Error-Tracking/AttributeError-MissingAttribute-Error-application.md

# AttributeError-MissingAttribute-Error-application

## Meaning

Missing attribute error occurs (triggering Sentry error issue with exception type AttributeError) because application attempts to access non-existent attribute, causing application operations to fail when object does not have expected attribute. Error events display in Sentry Issue Details page with error message patterns "attribute error", "has no attribute", stack traces show application code layers, and error levels indicate Error or Warning severity. This affects the application layer and indicates object attribute issues typically caused by missing object attributes, incorrect object types, or attribute access errors; application operations fail; if errors correlate with code changes, object structure problems may be visible in code modification history.

## Impact

Sentry error issue alerts fire; missing attribute errors occur; application operations fail; applications return attribute errors; users affected; issues remain in New or Ongoing status; error levels show Error or Warning severity; stack traces indicate attribute access failures; error events display in Sentry Issue Details page. Object attribute access fails; affected user count grows; error count increases continuously; issues show Medium or Low priority in Sentry dashboard; application functionality degrades; attribute errors occur. Attribute errors increase; object structure problems occur; if errors correlate with code changes, object structure problems may cause application failures.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify AttributeError with "attribute error" or "has no attribute" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected application files and attribute access location - Analyze event frequency over last 24 hours

(constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract attribute name from error message (`metadata.value`) matching pattern "has no attribute [attribute_name]" to identify missing object attribute and the object type that should have it.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. List similar issues for project `<project-name>` and check if they share common release or deployment timestamps to identify broader incidents.

## Diagnosis

1. Analyze the stack trace frames from the most recent event (`entries[].data.values[].stacktrace.frames[]`) and error message (`metadata.value`) matching pattern "has no attribute [attribute_name]" to identify the object type and missing attribute. If the stack trace shows the error

## 01-Error-Tracking/ConnectionError-ConnectionRefused-Database-Error-application.md

# ConnectionError-ConnectionRefused-Database-Error-application

## Meaning

Database connection error occurs (triggering Sentry error issue with exception type ConnectionError) because database server connection is refused, causing database operations to fail when attempting to connect to database server. Error events display in Sentry Issue Details page with error message patterns "connection refused", "cannot connect to database", stack traces show database connection layer (sentry/db/postgres/base.py or similar), and error levels indicate Error or Fatal severity. This affects the database layer and indicates database connectivity issues

typically caused by database service down, network connectivity problems, or database configuration errors; database operations fail.

## Impact

Sentry error issue alerts fire; database connection fails; database operations fail; applications return database errors; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate connection failures; error events display in Sentry Issue Details page. Database unavailability causes complete application failure; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades completely. Database errors increase; all database-dependent operations fail.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify ConnectionError with "connection refused" pattern and database component (filename contains "db", "postgres", "mysql", "database").

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected database connection files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract database connection details from error message ( `metadata.value` ) including host, port, and database name to identify target database instance.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze database configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify database connection attempts, connection failures, and network errors.

7. List similar issues for project `<project-name>` and check if they share common release or deployment timestamps to identify broader inci

## 01-Error-Tracking/ConnectionError-ConnectionRefused-Redis-Error-application.md

# ConnectionError-ConnectionRefused-Redis-Error-application

## Meaning

Redis connection error occurs (triggering Sentry error issue with exception type ConnectionError) because Redis server connection is refused, causing cache and queue operations to fail when attempting to connect to Redis on port 6379. Error events display in Sentry Issue Details page with error message patterns "connection refused", "Error 111 connecting to 127.0.0.1:6379", stack traces show queue or cache layer (sentry/monitoring/queues.py or similar), and error levels indicate Error or Fatal severity. This affects the cache and queue layer and indicates Redis connectivity issues typically caused by Redis service down, network connectivity problems, or Redis configuration errors; cache operations fail.

## Impact

Sentry error issue alerts fire; Redis connection fails; cache operations fail; queue operations fail; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate connection failures; error events display in Sentry Issue Details page. Cache unavailability causes performance degradation; queue processing stops; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades. Redis errors increase; cache misses occur; queue backlogs form.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify ConnectionError with "connection refused" pattern and Redis component (port 6379 or queues.py filename).

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected queue or cache files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract Redis connection details from error message ( `metadata.value` ) including host (127.0.0.1 or hostname) and port (6379) to identify target Redis instance.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze Redis configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify Redis connection attempts, connection failures, and queue operation errors.

7. List similar issues for project `<project-name>` and check if they share common release or deployment timestamps to identify broader incidents.

8. List tags for issue `<i

## 01-Error-Tracking/ConsumerError-ConnectionError-Kafka-Error-application.md

# ConsumerError-ConnectionError-Kafka-Error-application

## Meaning

Kafka consumer error occurs (triggering Sentry error issue with exception type ConsumerError) because Kafka broker connection error exists, causing event processing and message queue operations to fail when attempting to connect to Kafka brokers. Error events display in Sentry Issue Details page with error message patterns "connection error", "broker error", stack traces show Kafka consumer layer (sentry/utils/kafka.py or sentry/consumers/synchronized.py), and error levels indicate Error or Fatal severity. This affects the message queue and event processing layer and indicates Kafka broker connectivity issues typically caused by broker service down, network connectivity problems, or Kafka broker configuration errors; event processing fails.

## Impact

Sentry error issue alerts fire; Kafka consumer operations fail; event processing stops; message queue operations fail; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate Kafka broker connection failures; error events display in Sentry Issue Details page. Event processing pipeline breaks; downstream services affected; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades. Kafka errors increase; event backlog forms; data pipeline interruptions occur.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify ConsumerError with "connection error" or "broker error" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected Kafka consumer files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract Kafka broker connection details from error message ( `metadata.value` ) including broker host and port to identify target Kafka broker instance.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details

and check if commit is in PR. If PR found, retrieve PR diff to analyze Kafka broker configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify Kafka consumer logs, broker connection attempts, and network errors.

7. List similar issues for project `<project-name>` and check if they share common release or deployment timestamps to identify broader incidents.

8. List ta

## 01-Error-Tracking/ConsumerError-PartitionError-Kafka-Error-application.md

# ConsumerError-PartitionError-Kafka-Error-application

## Meaning

Kafka consumer error occurs (triggering Sentry error issue with exception type ConsumerError) because Kafka partition error exists, causing event processing and message queue operations to fail when attempting to consume from unavailable or misconfigured partitions. Error events display in Sentry Issue Details page with error message patterns "partition", "partition error", "partition not found", stack traces show Kafka consumer layer (sentry/utils/kafka.py or sentry/consumers/synchronized.py), and error levels indicate Error or Fatal severity. This affects the message queue and event processing layer and indicates Kafka partition configuration issues typically caused by partition deletion, partition rebalancing, or Kafka broker configuration problems; event processing fails.

## Impact

Sentry error issue alerts fire; Kafka consumer operations fail; event processing stops; message queue operations fail; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate Kafka partition failures; error events display in Sentry

Issue Details page. Event processing pipeline breaks; downstream services affected; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades. Kafka errors increase; event backlog forms; data pipeline interruptions occur.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify ConsumerError with "partition", "partition error", or "partition not found" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected Kafka consumer files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract Kafka partition details from error message ( `metadata.value` ) including topic name and partition number to identify affected partition.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze Kafka partition configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify Kafka consumer logs, partition assignment errors, and broker connection issues.

7. List similar issues for project `<project-name>` and check if they share common release or deployment

## 01-Error-Tracking/ConsumerError-TopicNotFound-Kafka-Error-application.md

# ConsumerError-TopicNotFound-Kafka-Error-application

## Meaning

Kafka consumer error occurs (triggering Sentry error issue with exception type ConsumerError) because Kafka topic is not found or partition is unavailable, causing event processing and message queue operations to fail when attempting to consume from non-existent topics. Error events display in Sentry Issue Details page with error message patterns "UNKNOWN_TOPIC_OR_PART", "Subscribed topic not available", stack traces show Kafka consumer layer (sentry/utils/kafka.py or sentry/consumers/synchronized.py), and error levels indicate Error or Fatal severity. This affects the message queue and event processing layer and indicates Kafka topic configuration issues typically caused by missing topic creation, topic deletion, or Kafka broker configuration problems; event processing fails.

## Impact

Sentry error issue alerts fire; Kafka consumer operations fail; event processing stops; message queue operations fail; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate Kafka consumer failures; error events display in Sentry Issue Details page. Event processing pipeline breaks; downstream services affected; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades. Kafka errors increase; event backlog forms; data pipeline interruptions occur.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify ConsumerError with "UNKNOWN_TOPIC_OR_PART" or "topic not available" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected Kafka consumer files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract Kafka topic name from error message (`metadata.value`) matching pattern "Subscribed topic not available: [topic_name]" or "UNKNOWN_TOPIC_OR_PART" to identify missing or unavailable Kafka topic.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze Kafka configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify Kafka consumer logs, topic subscription attempts, and broker connection errors.

7. List similar issues for project `<project-name>` and check if

## 01-Error-Tracking/DatabaseConnectionError-ConnectionRefused-Database-Error-application.md

# DatabaseConnectionError-ConnectionRefused-Database-Error-application

## Meaning

Database connection error occurs (triggering Sentry error issue with exception type DatabaseConnectionError) because database server connection is refused, causing database operations to fail when attempting to connect to database server. Error events display in Sentry Issue Details page with error message patterns "database connection refused", "cannot connect to database", stack traces show database connection layer (sentry/db/postgres/base.py or similar), and error levels indicate Error or Fatal severity. This affects the database layer and indicates database connectivity issues typically caused by database service down, network connectivity problems, or database configuration errors; database operations fail.

## Impact

Sentry error issue alerts fire; database connection fails; database operations fail; applications return database errors; users affected; issues remain in New or Ongoing status; error levels show Error or Fatal severity; stack traces indicate database connection failures; error events display in Sentry Issue Details page. Database unavailability causes complete application failure; affected user count grows; error count increases continuously; issues show High priority in Sentry dashboard; application functionality degrades completely. Database errors increase; all database-dependent operations fail.

## Playbook

1. Inspect available issue details including issue type ( `metadata.type` ), priority ( `priority` ), affected users ( `userCount` ), and error message ( `metadata.value` ) to verify DatabaseConnectionError with "database connection refused" or "cannot connect to database" pattern.

2. List events for issue `<issue-id>` and analyze event patterns: - Retrieve most recent event to inspect stack trace frames ( `entries[].data.values[].stacktrace.frames[]` ) and identify affected database connection files - Analyze event frequency over last 24 hours (constant, spike, gradual increase) and check user impact distribution (all users vs specific segments) from event data

3. Extract database connection details from error message ( `metadata.value` ) including host, port, and database name to identify target database instance.

4. Compare release timeline from issue details (firstRelease vs lastRelease if available) to check if issue correlates with deployments.

5. If `firstRelease.lastCommit` is populated, extract repository name from `firstRelease.versionInfo.package` or use service mapping, then retrieve GitHub commit details and check if commit is in PR. If PR found, retrieve PR diff to analyze database configuration changes.

6. If error context from Sentry is insufficient, search ELK logs for service around error timestamp to identify database connection attempts, connection failures, and network errors.

7. List similar issues for project `<project-name>` and check if they share common release or deployment timestamps to identify broader in